

AIAA 3111 - 数据挖掘导论

模式挖掘：高级方法

Dr. Hao Liu

AIAA 3111 - Introduction to Data Mining

Pattern Mining: Advanced Methods

Dr. Hao Liu

模式挖掘：高级方法

☐ 挖掘多层与多维模式



☐ 基于约束的模式挖掘

☐ 挖掘序列模式

☐ 挖掘子图模式

☐ 小结

Pattern Mining: Advanced Methods

☐ Mining Multi-level and multi-dimension Patterns



☐ Constraint-based Pattern Mining

☐ Mining Sequential Patterns

☐ Mining Subgraph Patterns

☐ Summary

挖掘多层频繁模式

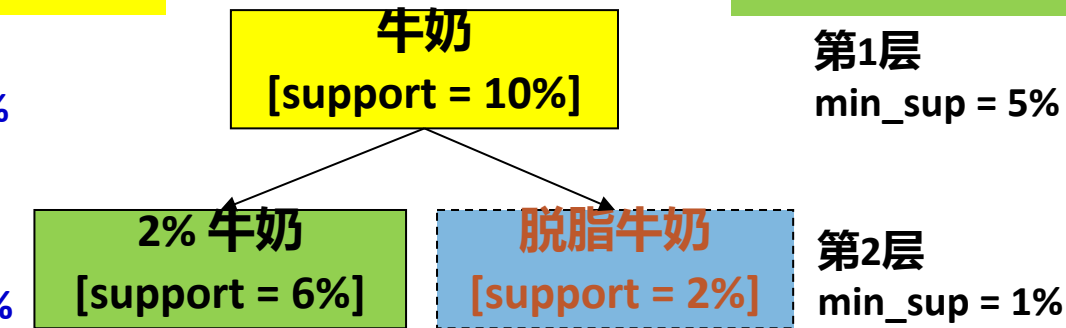
项通常形成层次结构

- 牛奶: Dairyland 2% 牛奶
- 面包: Wonder 全麦面包

统一支持度

第1层
min_sup = 5%

第2层
min_sup = 5%



递减支持度

第1层
min_sup = 5%

第2层
min_sup = 1%

按概念层次为项设置最小支持度阈值

- 多层使用统一 min-support (合理吗?)
- 层递减 min-support:
 - 较低层的项预期支持度更低
- 高效挖掘: 共享多层挖掘
 - 用最低 min-support 向下传递候选项集

Mining Multiple-Level Frequent Patterns

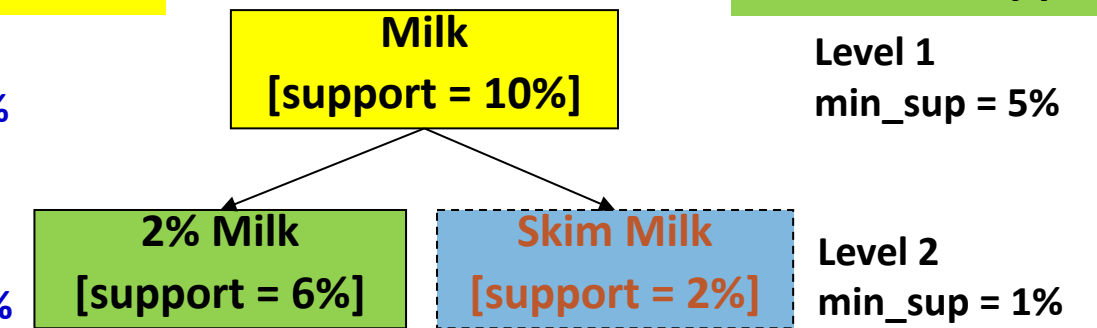
Items often form hierarchies

- Milk: Dairyland 2% milk
- Bread: Wonder wheat bread

Uniform support

Level 1
min_sup = 5%

Level 2
min_sup = 5%



Reduced support

Level 1
min_sup = 5%

Level 2
min_sup = 1%

Min-support thresholds for items following concept hierarchies

- Uniform min-support across multiple levels (reasonable?)
- Level-reduced min-support:
 - Items at the lower level are expected to have lower support
- Efficient mining: *Shared* multi-level mining
 - Use the lowest min-support to pass down the set of candidates

挖掘多层关联时的冗余过滤

- ❑ 冗余过滤：因“祖先”关系而冗余
 - ❑ 牛奶 $\Rightarrow$ 全麦面包 [support = 8%, confidence = 70%] (1)
 - ❑ 2% 牛奶 $\Rightarrow$ 全麦面包 [support = 2%, confidence = 72%] (2)
 - ❑ 假设 2% 牛奶销量约为牛奶总销量（按加仑计）的 $\frac{1}{4}$
 - ❑ (2) 应能从 (1) “导出”
- ❑ 一般地：若某规则的支持度接近其“祖先”规则给出的“期望”值，且置信度与其“祖先”相近，则该规则是冗余的

问：若规则 (1) 是规则 (2) 的祖先，应剪枝哪一条？

Redundancy Filtering at Mining Multi-Level Associations

- ❑ Redundancy filtering: redundant due to “ancestor” relationships
 - ❑ milk $\Rightarrow$ wheat bread [support = 8%, confidence = 70%] (1)
 - ❑ 2% milk $\Rightarrow$ wheat bread [support = 2%, confidence = 72%] (2)
 - ❑ Suppose the 2% milk sold is about $\frac{1}{4}$ of milk sold in gallons
 - ❑ (2) should be able to be “derived” from (1)
- ❑ In general: A rule is *redundant* if its support is close to the “expected” value, according to its “ancestor” rule, and it has a similar confidence as its “ancestor”

Q: If rule (1) is an ancestor of rule (2), which one to prune?

针对不同类项的定制最小支持度

- 以往在每次关联挖掘中，对所有待挖掘的项或项集使用同一 min-support 阈值
- 实际中，有些项（如 diamond、watch、...）价值高但出现频率低
- 因此有必要为不同类项定制 min-support
- 一种方法：使用分组的“个性化” min-support
 - 例如 {diamond, watch}: 0.05%; {bread, milk}: 5%; ...
 - 如何高效挖掘此类规则？
 - 现有可扩展挖掘算法可较容易地推广到这类情形

Customized Min-Supports for Different Kinds of Items

- We have used the same min-support threshold for all the items or item sets to be mined in each association mining
- In reality, some items (e.g., diamond, watch, ...) are valuable but less frequent
- It is necessary to have customized min-support settings for different kinds of items
- One Method: Use **group-based “individualized” min-support**
 - E.g., {diamond, watch}: 0.05%; {bread, milk}: 5%; ...
 - How to mine such rules efficiently?
 - Existing scalable mining algorithms can be easily extended to cover such cases

挖掘多维关联

- ❑ 单维规则（例如，项均属于“商品”维）
 - ❑ $\text{buys}(X, \text{“milk”}) \Rightarrow \text{buys}(X, \text{“bread”})$
- ❑ 多维规则（即项属于 ≥ 2 个维或谓词）
 - ❑ 维间关联规则（谓词不重复）
 - ❑ $\text{age}(X, \text{“18-25”}) \Rightarrow \text{occupation}(X, \text{“student”}) \Rightarrow \text{buys}(X, \text{“coke”})$
 - ❑ 混合维关联规则（谓词可重复）
 - ❑ $\text{age}(X, \text{“18-25”}) \Rightarrow \text{buys}(X, \text{“popcorn”}) \Rightarrow \text{buys}(X, \text{“coke”})$
- ❑ 属性可以是分类型或数值型
 - ❑ 分类型属性（如职业、商品：取值之间无序）：用数据立方体挖掘维间关联
 - ❑ 定量属性：数值型，取值之间有隐含序—离散化、聚类与梯度方法

Mining Multi-Dimensional Associations

- ❑ Single-dimensional rules (e.g., items are all in “product” dimension)
 - ❑ $\text{buys}(X, \text{“milk”}) \Rightarrow \text{buys}(X, \text{“bread”})$
- ❑ Multi-dimensional rules (i.e., items in ≥ 2 dimensions or predicates)
 - ❑ Inter-dimension association rules (*no repeated predicates*)
 - ❑ $\text{age}(X, \text{“18-25”}) \wedge \text{occupation}(X, \text{“student”}) \Rightarrow \text{buys}(X, \text{“coke”})$
 - ❑ Hybrid-dimension association rules (*repeated predicates*)
 - ❑ $\text{age}(X, \text{“18-25”}) \wedge \text{buys}(X, \text{“popcorn”}) \Rightarrow \text{buys}(X, \text{“coke”})$
- ❑ Attributes can be categorical or numerical
 - ❑ Categorical Attributes (e.g., *profession, product*: no ordering among values): Data cube for inter-dimension association
 - ❑ Quantitative Attributes: Numeric, implicit ordering among values—discretization, clustering, and gradient approaches

稀有模式与负模式

- ❑ 稀有模式
 - ❑ 支持度很低但有趣（例如购买 Rolex 手表）
 - ❑ 如何挖掘？为不同项组设置个性化、基于分组的 min-support 阈值
- ❑ 负模式
 - ❑ 负相关：不太可能同时出现
 - ❑ 例：同一顾客同时购买 Ford Expedition（SUV）和 Ford Fusion（混合动力车）的可能性很低，因此购买 Ford Expedition 与购买 Ford Fusion 很可能是负相关模式
 - ❑ 如何定义负模式？

Rare Patterns vs. Negative Patterns

- ❑ Rare patterns
 - ❑ Very low support but interesting (e.g., buying Rolex watches)
 - ❑ How to mine them? Setting individualized, group-based min-support thresholds for different groups of items
- ❑ Negative patterns
 - ❑ Negatively correlated: Unlikely to happen together
 - ❑ Ex.: Since it is unlikely that the same customer buys both a **Ford Expedition** (an SUV car) and a **Ford Fusion** (a hybrid car), buying a **Ford Expedition** and buying a **Ford Fusion** are likely negatively correlated patterns
 - ❑ How to define negative patterns?

负相关模式的定义

- 基于支持度的定义
 - 若项集 A 与 B 都频繁但很少一起出现, 即
 - $\text{sup}(A \cup B) \ll \text{sup}(A) \times \text{sup}(B)$
 - 则 A 与 B 负相关
- 这对大规模事务数据集是好定义吗?
- 例: 假设商店中针线包 A 和 B 各售出 100 次, 但仅有一笔事务同时包含 A 和 B
 - 当总共有 200 笔事务时,
 - $s(A \cup B) = 0.005, s(A) \times s(B) = 0.25, s(A \cup B) \ll s(A) \times s(B)$
 - 但当共有 105 笔事务时,
 - $s(A \cup B) = 1/105, s(A) \times s(B) = 1/103 \times 1/103, s(A \cup B) > s(A) \times s(B)$
- 问题出在哪里?
 - 零事务: 基于支持度的定义不具有零事务不变性!

这是否让你联想到提升度 (lift) 的定义?

Defining Negative Correlated Patterns

- A support-based definition
 - If itemsets A and B are both frequent but rarely occur together, i.e.,
 - $\text{sup}(A \cup B) \ll \text{sup}(A) \times \text{sup}(B)$
 - Then A and B are negatively correlated
- Is this a good definition for large transaction datasets?
- Ex.: Suppose a store sold two needle packages A and B 100 times each, but only one transaction contained both A and B
 - When there are in total 200 transactions, we have
 - $s(A \cup B) = 0.005, s(A) \times s(B) = 0.25, s(A \cup B) \ll s(A) \times s(B)$
 - But when there are 10^5 transactions, we have
 - $s(A \cup B) = 1/10^5, s(A) \times s(B) = 1/10^3 \times 1/10^3, s(A \cup B) > s(A) \times s(B)$
- What is the problem here?
 - Null transactions: The support-based definition is not null-invariant!

Does this remind you the definition of lift?

定义负相关：定义需具备零事务不变性

- 好的负相关定义应当具有零事务不变性！
 - 直观：项集 A 与 B 是否负相关，不应受零事务数量影响
- 基于 Kulczynski 度量的定义
 - 若项集 A 与 B 频繁且 $(s(A \cup B)/s(A) + s(A \cup B)/s(B))/2 < \epsilon$, 则 A 与 B 负相关
- 对同一针线包问题：
 - 无论总共有 200 还是 105 笔事务
 - 若 $\epsilon = 0.01$, 则

$$(s(A \cup B)/s(A) + s(A \cup B)/s(B))/2 = (0.01 + 0.01)/2 < \epsilon$$

ϵ 为负
模式阈值

Defining Negative Correlation: Need Null-Invariance in Definition

- A good definition on negative correlation should be null-invariant!
 - Intuition: Whether two itemsets A and B are negatively correlated should not be influenced by the number of null-transactions
- A Kulczynski measure-based definition
 - If itemsets A and B are frequent but $(s(A \cup B)/s(A) + s(A \cup B)/s(B))/2 < \epsilon$, then A and B are negatively correlated
- For the same needle package problem:
 - No matter there are in total 200 or 10^5 transactions
 - If $\epsilon = 0.01$, we have

ϵ is a negative
pattern threshold

模式挖掘：高级方法

❑ 挖掘多层与多维模式

❑ 基于约束的模式挖掘



❑ 挖掘序列模式

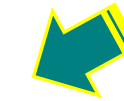
❑ 挖掘子图模式

❑ 小结

Pattern Mining: Advanced Methods

❑ Mining Multi-level and multi-dimension Patterns

❑ Constraint-based Pattern Mining



❑ Mining Sequential Patterns

❑ Mining Subgraph Patterns

❑ Summary

为何需要基于约束的挖掘？

- 实践中的模式挖掘：往往是用户引导的交互过程
 - 用户通过数据挖掘查询语言（或图形界面）指定各类约束，引导挖掘内容
- 什么是基于约束的挖掘？
 - 在挖掘过程中一并使用用户提供的约束
- 为何需要基于约束的挖掘？
 - 用户灵活性：用户指定要挖掘什么
 - 优化：系统利用这些约束提高挖掘效率
 - 例如，将约束深度下推到挖掘过程中

Why Constraint-Based Mining?

- Pattern mining in practice: Often a user-guided, **interactive** process
 - User directs what to be mined using a **data mining query language** (or a graphical user interface), **specifying various kinds of constraints**
- What is constraint-based mining?
 - Mine together with user-provided constraints
- Why constraint-based mining?
 - User flexibility: User provides **constraints** on what to be mined
 - Optimization: System explores such constraints for mining efficiency
 - E.g., Push constraints deeply into the mining process

数据挖掘中用户指定的各类约束

- 知识类型约束—指定要挖掘何种知识
 - 例如：分类、关联、聚类、离群点发现、...
- 数据约束—使用类 SQL 查询
 - 例如：找出今年纽约门店中一起售出的商品
- 维/层约束—类似于关系数据库中的投影
 - 例如：与地区、价格、品牌、顾客类别相关
- 兴趣度约束—各类阈值
 - 例如：强规则：min_sup $\geq$ 0.02, min_conf $\geq$ 0.6, min_correlation $\geq$ 0.7
- 规则（或模式）约束  本次讨论的重点
 - 例如：小额销售 (price < \$10) 引发大额销售 (sum > \$200)

Various Kinds of User-Specified Constraints in Data Mining

- Knowledge type constraint—Specifying what kinds of knowledge to mine
 - E.g.: Classification, association, clustering, outlier finding, ...
- Data constraint—using SQL-like queries
 - E.g.: Find products sold together in NY stores this year
- Dimension/level constraint—similar to projection in relational database
 - E.g.: In relevance to region, price, brand, customer category
- Interestingness constraint—various kinds of thresholds
 - E.g.: Strong rules: min_sup $\geq$ 0.02, min_conf $\geq$ 0.6, min_correlation $\geq$ 0.7
- Rule (or pattern) constraint  The focus of this study
 - E.g.: Small sales (price < \$10) triggers big sales (sum > \$200)

利用模式反单调性剪枝模式空间

TID	事务
10	a, b, c, d, f, h
20	b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

项	价格	利润
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-10
g	80	20
h	10	5

- 约束 c 是反单调的
 - 若项集 S 违反约束 c ，则其任意超集也违反 c
 - 即：对项集 S 的挖掘可以终止
- 例 1： $c_1: \text{sum}(S.\text{price}) \leq v$ 是反单调的
- 例 2： $c_2: \text{range}(S.\text{profit}) \leq 15$ 是反单调的
 - 项集 ab 违反 c_2 ($\text{range}(ab) = 40$)
 - ab 的每个超集亦然
- 例 3. $c_3: \text{sum}(S.\text{Price}) \geq v$ 不是反单调的
- 例 4. $c_4: \text{support}(S) \geq \sigma$ 是反单调的吗？
 - 是的！Apriori 剪枝本质上就是用反单调约束进行剪枝！

注：item.price > 0
利润可为负

Pattern Space Pruning with Pattern Anti-Monotonicity

TID	Transaction
10	a, b, c, d, f, h
20	b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

Item	Price	Profit
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-10
g	80	20
h	10	5

- A constraint c is **anti-monotone**
 - If an itemset S **violates** constraint c , so does any of its superset
 - That is, mining on itemset S can be terminated
- Ex. 1: $c_1: \text{sum}(S.\text{price}) \leq v$ is **anti-monotone**
- Ex. 2: $c_2: \text{range}(S.\text{profit}) \leq 15$ is **anti-monotone**
 - Itemset ab violates c_2 ($\text{range}(ab) = 40$)
 - So does every superset of ab
- Ex. 3. $c_3: \text{sum}(S.\text{Price}) \geq v$ is **not anti-monotone**
- Ex. 4. Is $c_4: \text{support}(S) \geq \sigma$ anti-monotone?
 - Yes! Apriori pruning is essentially pruning with an anti-monotonic constraint!

Note: item.price > 0
Profit can be negative

模式单调性及其作用

TID	事务
10	a, b, c, d, f, h
20	b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

项	价格	利润
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-10
g	80	20
h	10	5

- 约束 c 是单调的：若项集 S 满足约束 c ，则其任意超集也满足 c
 - 即：后续挖掘不必再检查 c
 - 不如反单调那么有益
- 例 1: $c_1: \text{sum}(S.\text{Price}) \geq v$ 是单调的
- 例 2: $c_2: \min(S.\text{Price}) \geq v$ 是单调的
- 例 3: $c_3: \text{range}(S.\text{profit}) \geq 15$ 是单调的
 - 项集 ab 满足 c_3
 - ab 的每个超集亦然

Pattern Monotonicity and Its Roles

TID	Transaction
10	a, b, c, d, f, h
20	b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

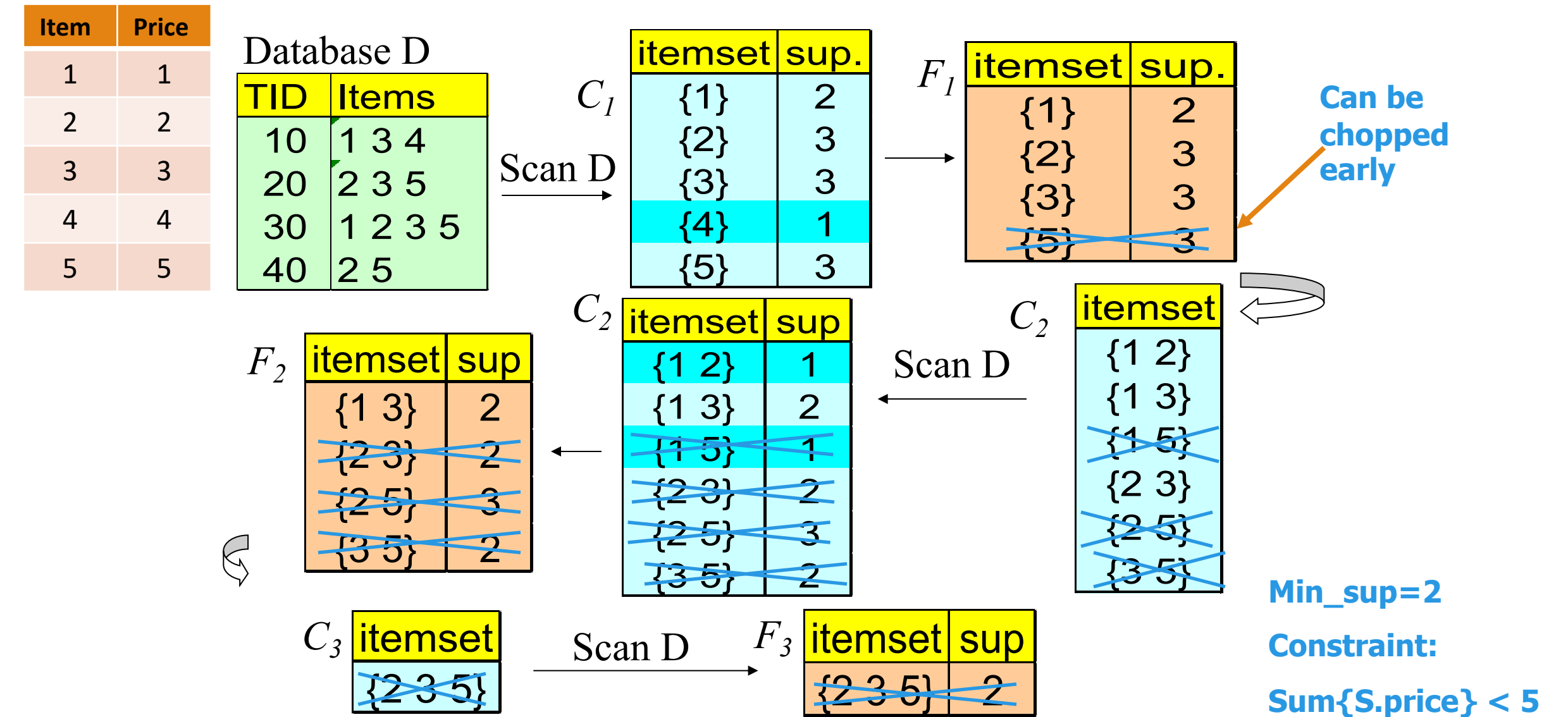
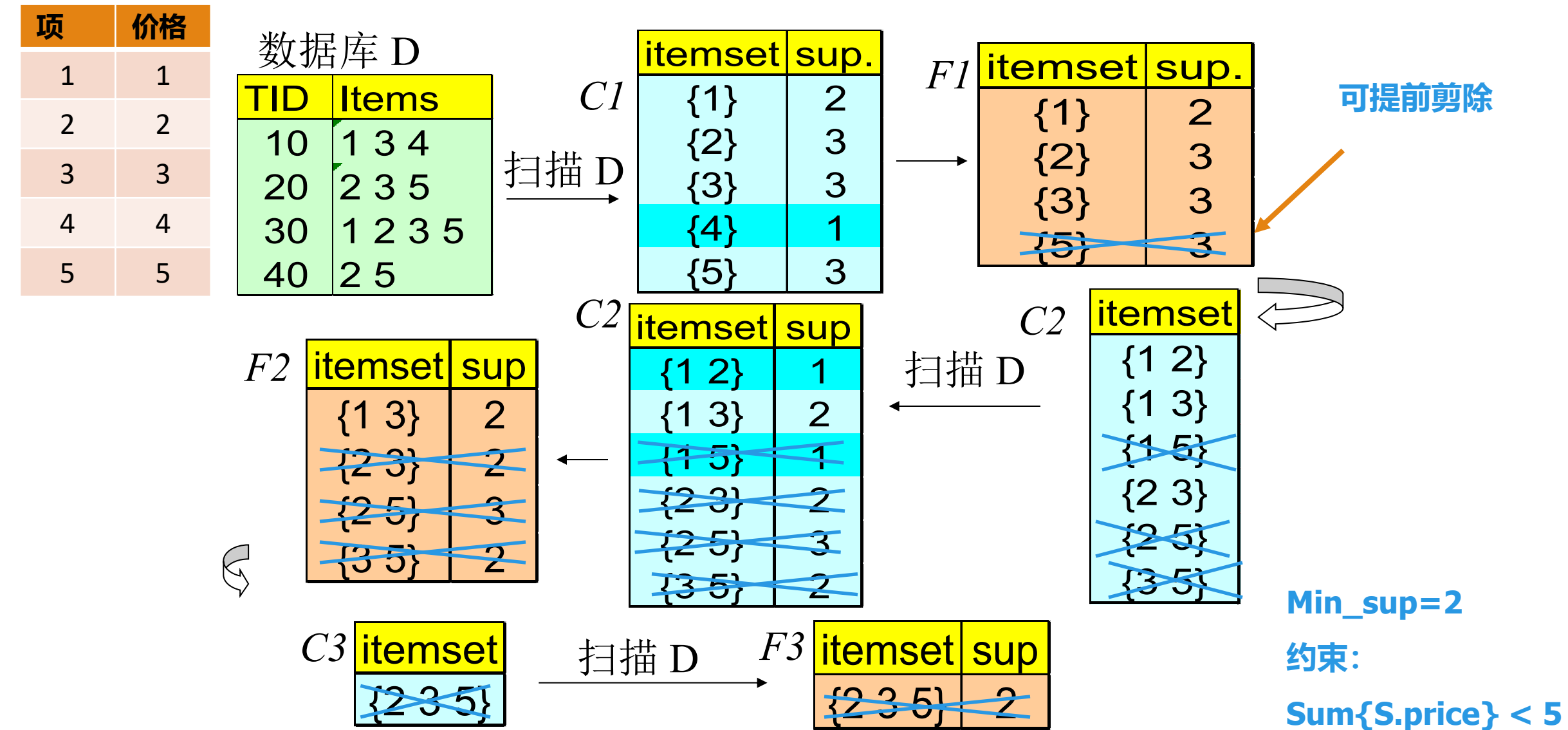
min_sup = 2

Item	Price	Profit
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-10
g	80	20
h	10	5

- A constraint c is *monotone*: If an itemset S **satisfies** the constraint c , so does any of its superset
 - That is, we do not need to check c in subsequent mining
 - Not as beneficial as anti-monotone
- E.g. 1: $c_1: \text{sum}(S.\text{Price}) \geq v$ is **monotone**
- E.g. 2: $c_2: \min(S.\text{Price}) \geq v$ is **monotone**
- E.g. 3: $c_3: \text{range}(S.\text{profit}) \geq 15$ is **monotone**
 - Itemset ab satisfies c_3
 - So does every superset of ab

面向模式反单调约束的 Apriori

Apriori for Pattern Anti-Monotone Constraint



可转换约束：对事务中的数据排序

TID	事务
10	a, b, c, d, f, h
20	a, b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

项	价格	利润
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-5
g	80	30
h	10	5

- 通过对事务中的项适当排序，可将困难约束转换为(反)单调约束
- 考察 $c_1: \text{avg}(S.\text{profit}) > 20$
 - 按(利润)取值降序排列项
 - $\langle a, g, h, b, f, d, c, e \rangle$
 - 项集 ab 违反 c_1 ($\text{avg}(ab) = 20$)
 - ab^* (即 ab -投影数据库)亦然
 - C_1 : 若模式按正确顺序增长, 则约束为反单调！

Convertible Constraints: Ordering Data in Transactions

TID	Transaction
10	a, b, c, d, f, h
20	a, b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

Item	Price	Profit
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-5
g	80	30
h	10	5

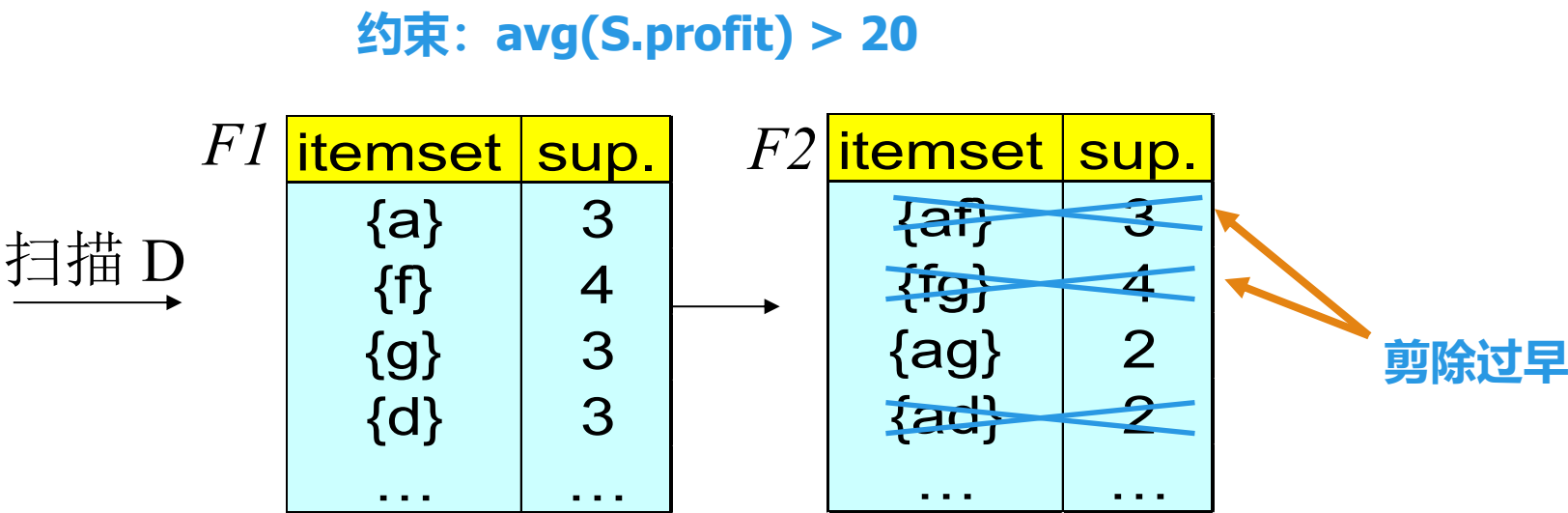
- Convert tough constraints into (anti-)monotone by proper ordering of items in transactions
- Examine $c_1: \text{avg}(S.\text{profit}) > 20$
 - Order items in (profit) value-descending order
 - $\langle a, g, h, b, f, d, c, e \rangle$
 - An itemset ab violates c_1 ($\text{avg}(ab) = 20$)
 - So does ab^* (i.e., ab -projected DB)
 - C_1 : **anti-monotone if patterns grow in the right order!**

项重排序对 Apriori 有效吗？

TID	事务
10	a, b, c, d, f, h
20	a, b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

项	价格	利润
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-5
g	80	30
h	10	5



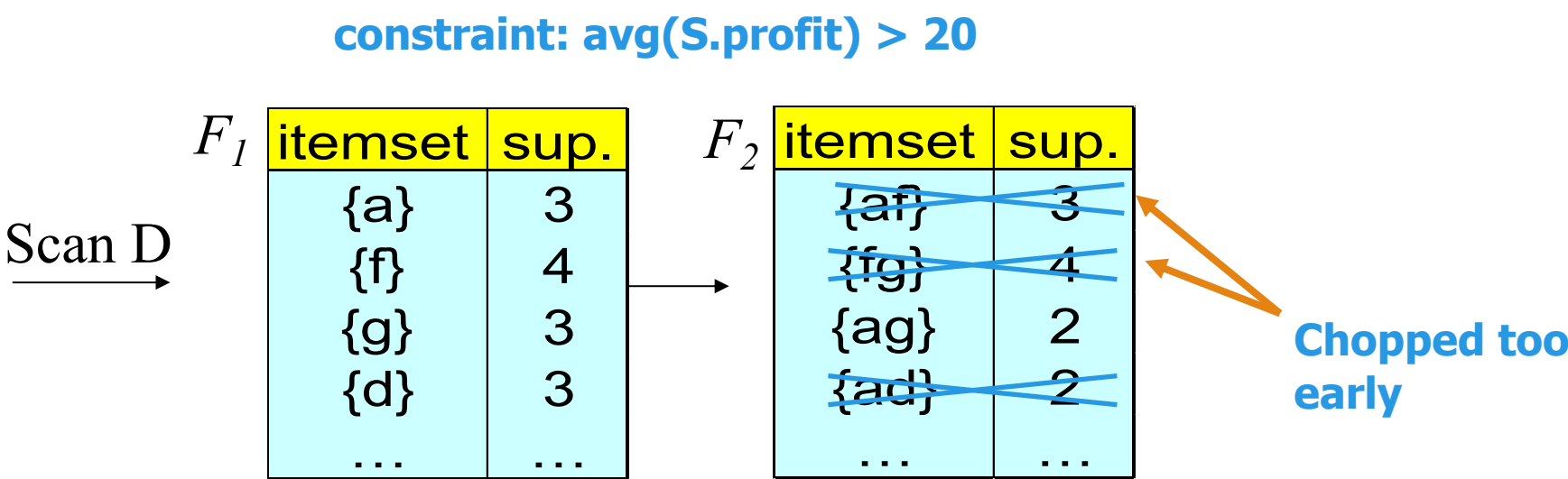
- $\text{avg}(gf) = 12.5 < 20$, $\text{avg}(af) = 17.5 < 20$, $\text{avg}(ag) = 35 > 20$
- 但 $\text{avg}(agf) = 21.7 > 20$
- Apriori 不会生成“agf”作为候选

Can item-reordering work for Apriori?

TID	Transaction
10	a, b, c, d, f, h
20	a, b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

Item	Price	Profit
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-5
g	80	30
h	10	5



- $\text{avg}(gf) = 12.5 < 20$, $\text{avg}(af) = 17.5 < 20$, $\text{avg}(ag) = 35 > 20$
- But $\text{avg}(agf) = 21.7 > 20$
- Apriori will not generate “agf” as a candidate

利用数据反单调性剪枝数据空间

TID	事务
10	a, b, c, d, f, h
20	b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

项	价格	利润
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-10
g	80	20
h	10	5

- 约束 c 是数据反单调的：挖掘过程中，若数据项 t 在 c 下无法满足模式 p ，则 t 也无法满足 p 的超集
 - 数据空间剪枝：可剪除数据项 t
- 例 1： $c_1: \text{sum}(S.\text{Profit}) \geq v$ 是数据反单调的
 - 令约束 c_1 为： $\text{sum}(S.\text{Profit}) \geq 25$
 - $T_{30}: \{b, c, d, f, g\}$ 可删除，因其任何组合都无法构成利润之和 ≥ 25 的 S
- 例 2： $c_2: \text{min}(S.\text{Price}) \geq v$ 是数据反单调的
 - 设 $v = 5$ ，但某事务（如 T_{50} ）中每项价格都高于 10
- 例 3： $c_3: \text{range}(S.\text{Profit}) > 25$ 是数据反单调的

Data Space Pruning with Data Anti-Monotonicity

TID	Transaction
10	a, b, c, d, f, h
20	b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

Item	Price	Profit
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-10
g	80	20
h	10	5

- A constraint c is **data anti-monotone**: In the mining process, if a data entry t cannot satisfy a pattern p under c , t cannot satisfy p 's superset either
 - Data space pruning: Data entry t can be pruned
- Ex. 1: $c_1: \text{sum}(S.\text{Profit}) \geq v$ is **data anti-monotone**
 - Let constraint c_1 be: $\text{sum}(S.\text{Profit}) \geq 25$
 - $T_{30}: \{b, c, d, f, g\}$ can be removed since none of their combinations can make an S whose sum of the profit is ≥ 25
- Ex. 2: $c_2: \text{min}(S.\text{Price}) \leq v$ is **data anti-monotone**
 - Consider $v = 5$ but every item in a transaction, say T_{50} , has a price higher than 10
- Ex. 3: $c_3: \text{range}(S.\text{Profit}) > 25$ is **data anti-monotone**

数据空间剪枝应递归进行

TID	事务	项	利润
10	a, b, c, d, f, h	a	40
20	b, c, d, f, g, h	b	0
30	b, c, d, f, g	c	-20
40	a, c, e, f, g	d	-15
min_sup = 2		e	-30
		f	-10
		g	20
		h	5

- 例: $c_3: \text{range}(S.\text{Profit}) > 25$
 - 检查 b 的投影数据库
 - 但项“a”不频繁 (sup = 1)

- 从 T10 中删除“a (40)”后
 - T10 不再能满足 c_3
 - 因为“b (0)”与“c (-20), d (-15), f (-10), h (5)”
 - 删除 T10 后, 也可剪除 T20 中的“h”

b 的投影数据库



TID	事务
10	a, c, d, f, h
20	c, d, f, g, h
30	c, d, f, g

b 的投影数据库

TID	事务
10	a, c, d, f, h
20	c, d, f, g, h
30	c, d, f, g

Data Space Pruning Should Be Explored Recursively

TID	Transaction	Item	Profit
10	a, b, c, d, f, h	a	40
20	b, c, d, f, g, h	b	0
30	b, c, d, f, g	c	-20
40	a, c, e, f, g	d	-15
min_sup = 2		e	-30
		f	-10
		g	20
		h	5

- Example. $c_3: \text{range}(S.\text{Profit}) > 25$
 - We check b's projected database
 - But item “a” is infrequent (sup = 1)

- After removing “a (40)” from T_{10}
 - T_{10} cannot satisfy c_3 any more
 - Since “b (0)” and “c (-20), d (-15), f (-10), h (5)”
 - By removing T_{10} , we can also prune “h” in T_{20}

b's-proj. DB

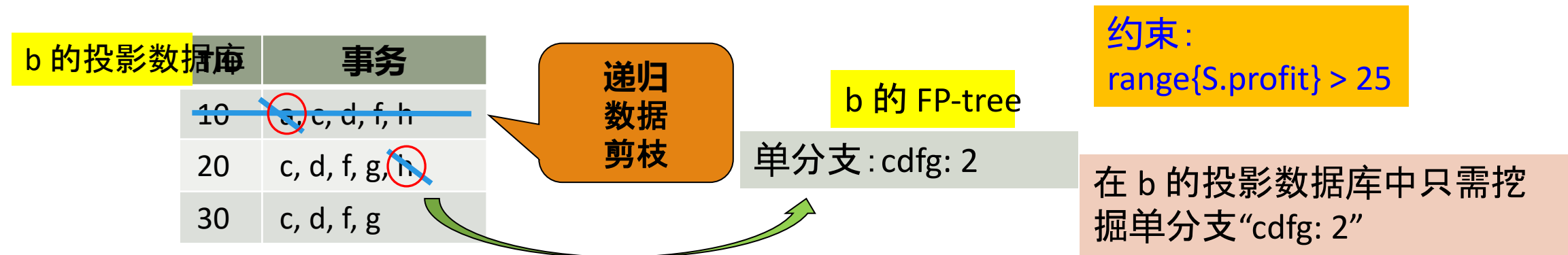


TID	Transaction
10	a, c, d, f, h
20	c, d, f, g, h
30	c, d, f, g

b's-proj. DB

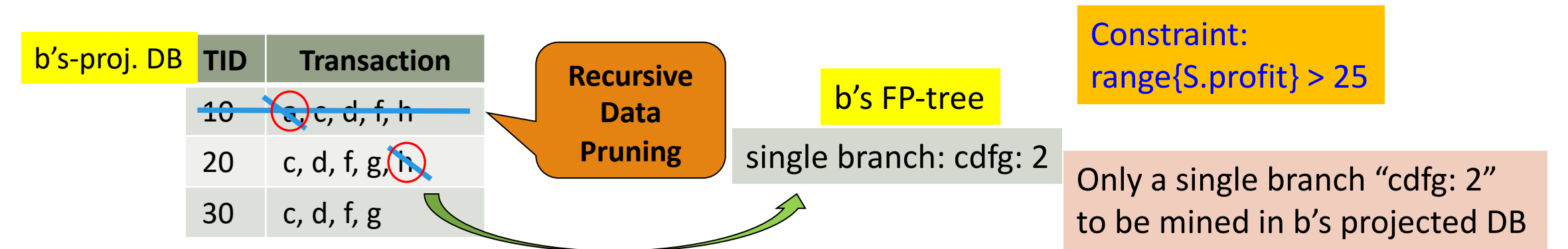
TID	Transaction
10	a, c, d, f, h
20	c, d, f, g, h
30	c, d, f, g

数据空间剪枝的递归探索（续）



- 注：仅当在 b 的投影数据库中（按 min-sup）剪除“a”之后，c3 才能有效剪除 T10

Data Space Pruning Explored Recursively (Continued)



- Note: c_3 prunes T_{10} effectively only after “a” is pruned (by min-sup) in b's projected DB

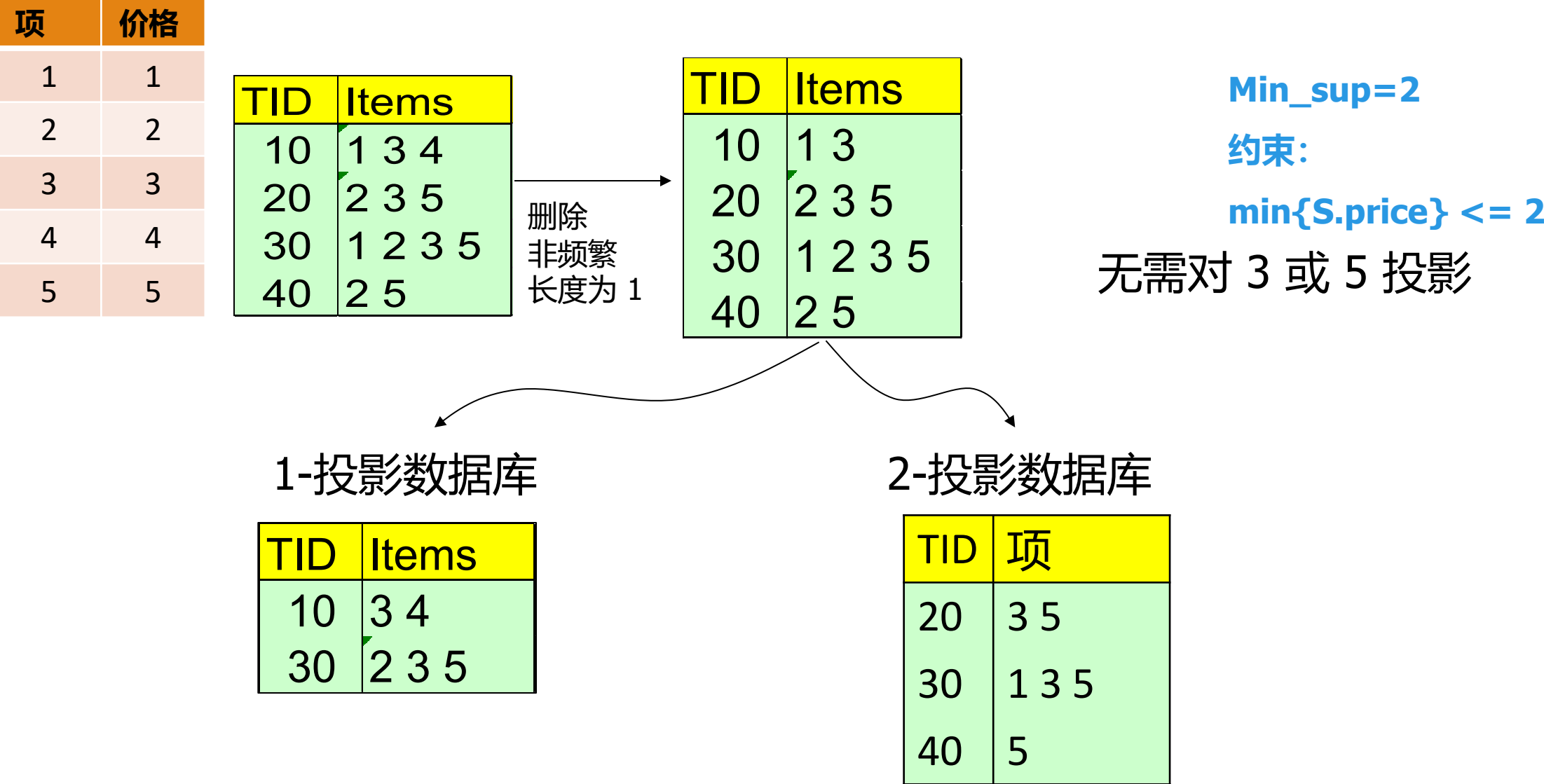
简洁性：同时剪枝数据空间与模式空间

- 简洁性：若约束 c 可通过直接操作数据来强制满足
- 例 1：找包含项 i 的模式
 - 只挖掘 i -投影数据库（数据空间剪枝）
- 例 2：找不含项 i 的模式
 - 从数据库中删除 i 再挖掘（模式空间剪枝）
- 例 3： $c_3: \min(S.Price) \leq v$ 是简洁的
 - 从价格 $\leq v$ 的项开始，并删除仅含高价项的事务（模式 + 数据空间剪枝）
- 例 4： $c_4: \sum(S.Price) \geq v$ 不是简洁的
 - 无法事先判定，因为项集 S 的价格之和会持续增加

Succinctness: Pruning Both Data and Pattern Spaces

- Succinctness: If the constraint c can be enforced by directly manipulating the data
- E.g. 1: To find those patterns containing item i
 - Mine only i -projected DB (data space pruning)
- E.g. 2: To find those patterns without item i
 - Remove i from DB and then mine (pattern space pruning)
- E.g. 3: $c_3: \min(S.Price) \leq v$ is succinct
 - Start with only items whose price $\leq v$ and remove transactions with high-price items only (pattern + data space pruning)
- E.g. 4: $c_4: \sum(S.Price) \geq v$ is not succinct
 - It cannot be determined beforehand since sum of the price of itemset S keeps increasing

带约束的 FP-Growth: 将简洁约束下推至深处



不同类型的约束对应不同的剪枝策略

■ 总之，约束可分为模式空间剪枝约束与数据空间剪枝约束

模式空间剪枝约束	数据空间剪枝约束
■ 反单调: 若违反约束 c, 则可终止对其进一步挖掘 ■ 单调: 若已满足 c, 则不必再检查 c ■ 可转换: 若处理时可将项适当排序, 则 c 可转换为单调或反单调 ■ 简洁: 若约束 c 可通过直接操作数据来强制满足	■ 数据简洁: 可在初始模式挖掘过程中剪枝数据空间 ■ 数据反单调: 若事务 t 不满足 c, 则可剪除 t 以减少数据处理量

Different Kinds of Constraints Lead to Different Pruning Strategies

■ In summary, constraints can be categorized as **pattern space pruning** constraints vs. **data space pruning** constraints

Pattern space pruning constraints	Data space pruning constraints
■ Anti-monotonic: If constraint c is violated, its further mining can be terminated ■ Monotonic: If c is satisfied, no need to check c again ■ Convertible: c can be converted to monotonic or anti-monotonic if items can be properly ordered in processing ■ Succinct: If the constraint c can be enforced by directly manipulating the data	■ Data succinct: Data space can be pruned at the initial pattern mining process ■ Data anti-monotonic: If a transaction t does not satisfy c, then t can be pruned to reduce data processing effort

如何处理多个约束？

- 在模式挖掘中使用多个约束是有益的
- 但不同约束可能需要相互冲突的项排序
 - 若 c_1 与 c_2 的排序存在冲突
 - 先对数据排序并强制其中一个约束(哪一个?)
 - 再在挖掘投影数据库时强制另一个约束
- 例如 c_1 : $\text{avg}(S.\text{profit}) > 20$, 以及 c_2 : $\text{avg}(S.\text{price}) < 50$
 - 假设 c_1 剪枝能力更强
 - 先按利润降序排序并使用 c_1
 - 对每个投影数据库, 按价格升序排列事务, 挖掘时使用 c_2

How to Handle Multiple Constraints?

- It is beneficial to use multiple constraints in pattern mining
- But different constraints may require potentially conflicting item-ordering
 - If there exists conflict ordering between c_1 and c_2
 - Try to sort data and enforce *one constraint* first (which one?)
 - Then enforce the other constraint when mining the projected databases
- E.g. c_1 : $\text{avg}(S.\text{profit}) > 20$, and c_2 : $\text{avg}(S.\text{price}) < 50$
 - Assume c_1 has more pruning power
 - Sort in profit descending order and use c_1 first
 - For each project DB, sort trans. in price ascending order and use c_2 at mining

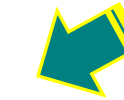
模式挖掘：高级方法

- ❑ 挖掘多层与多维模式
- ❑ 基于约束的模式挖掘
- ❑ 挖掘序列模式
- ❑ 挖掘子图模式
- ❑ 小结



Pattern Mining: Advanced Methods

- ❑ Mining Multi-level and multi-dimension Patterns
- ❑ Constraint-based Pattern Mining
- ❑ Mining Sequential Patterns
- ❑ Mining Subgraph Patterns
- ❑ Summary



序列模式挖掘

- 什么样的模式是序列的？
- 序列：次序确实重要
 - 不能交换序列中两个项而仍得到同一序列
- 例子：英语是序列的：主语 → 谓语 → 宾语
- 其他要点：
 - 对序列模式挖掘而言，不考虑项发生的具体时间
 - 时间序列分析则会考虑项发生的时间

Sequential Pattern Mining

- What kind of patterns are sequential?
- Sequential: The order really matters
 - You cannot swap two items in a sequence and have the same sequence
- Example: The English language is sequential: Subject → Verb → Object
- Other points:
 - For Sequential Pattern Mining, the time which the items occur is **not** considered
 - Time Series Analysis does take into account the time in which an item occurred

序列模式示例

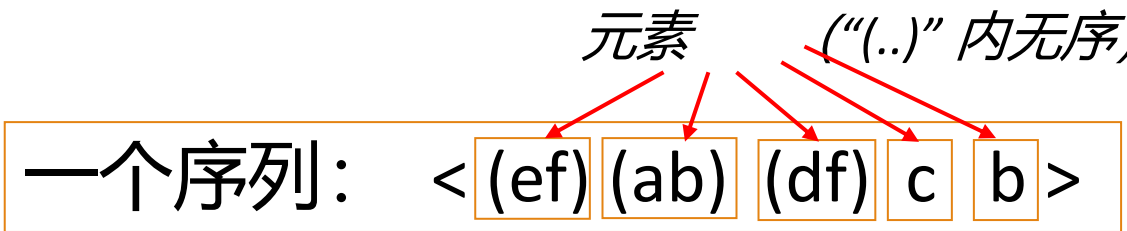
- 序列模式挖掘的应用
 - 顾客购物：先买笔记本电脑，再买数码相机，然后买智能手机
 - 医疗处理：去看医生、取药、医生监测进展、医生据此调整 → 增加/减少用药
 - 自然灾害：灾前、灾中、灾后
 - 科学实验：步骤 1、步骤 2、步骤 3
 - 股票市场：一组股票一起涨跌
 - 生物序列、DNA /蛋白质：若改变蛋白质的顺序，很可能得到不同的基因

Sequential Pattern Examples

- Application of Sequential pattern Mining
 - **Customer shopping:** Purchase a laptop first, then a digital camera, and then a smartphone
 - **Medical treatments:** Go to see a doctor, get drugs, doctor monitors progress, doctor reacts accordingly → more/less drugs
 - **Natural disasters:** Before the disaster, during the disaster, after the disaster
 - **Scientific Experiments:** Step 1, Step 2, Step 3
 - **Stocks Markets:** A set of stocks go up and down together
 - **Biological sequences, DNA /Protein:** If you change the order of a protein, it likely results in a different gene

序列模式与序列模式挖掘

- 序列模式挖掘：给定一组序列，找出频繁子序列的完全集（即满足 min_sup 阈值）



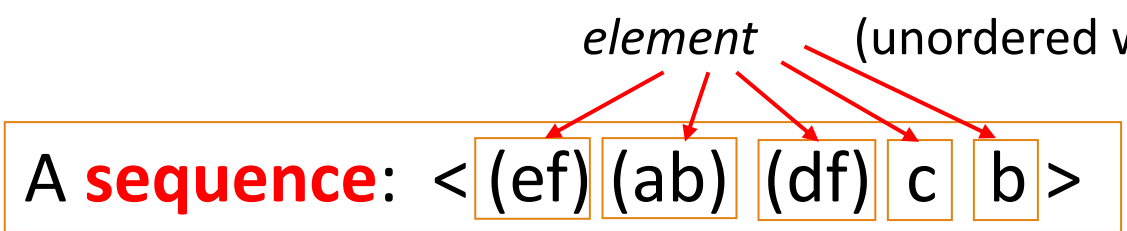
- 一个元素可包含一组项（也称为事件）
- * 元素内的项无序，我们按字母序列出

一个序列数据库

SID	序列
10	<a(abc)(ac)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(ab)(df)cb>
40	<eg(af)cbc>

Sequential Pattern and Sequential Pattern Mining

- Sequential pattern mining: Given a set of sequences, find the complete set of frequent subsequences (i.e., satisfying the min_sup threshold)



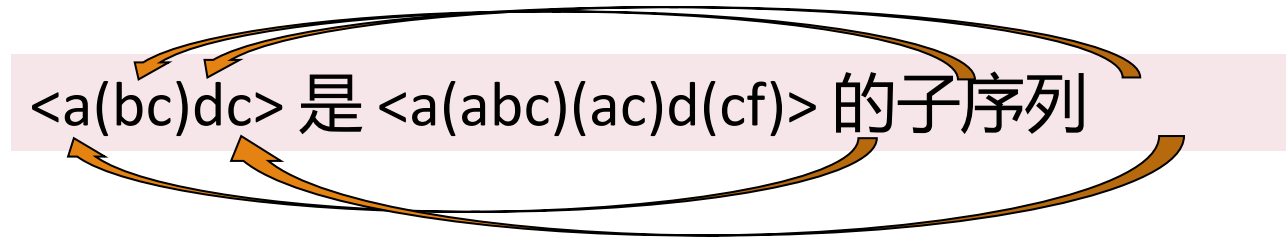
- An element may contain a set of items (also called events)
- * Items within an element are **unordered** and we list them alphabetically

A **sequence database**

SID	Sequence
10	<a(<u>ab</u> c)(a <u>c</u>)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(<u>ab</u>)(df) <u>c</u> b>
40	<eg(af)cbc>

序列模式与序列模式挖掘

- 序列模式挖掘：给定一组序列，找出频繁子序列的完全集（即满足 min_sup 阈值）



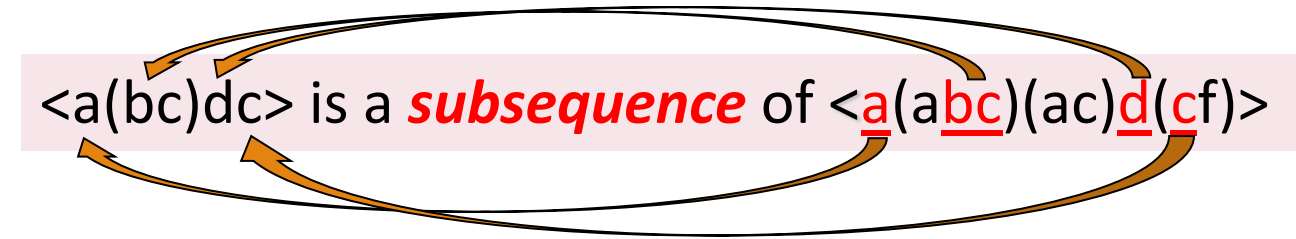
- 给定支持度阈值 min_sup = 2, <(ab)c> 是一个序列模式

一个序列数据库

SID	序列
10	<a(abc)(ac)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(ab)(df)cb>
40	<eg(af)cbc>

Sequential Pattern and Sequential Pattern Mining

- Sequential pattern mining: Given a set of sequences, find the **complete set of frequent subsequences** (i.e., satisfying the min_sup threshold)



- Given support threshold $min_sup = 2$, <(ab)c> is a **sequential pattern**

A **sequence database**

SID	Sequence
10	<a(abc)(a c)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(ab)(df) c b>
40	<eg(af)cbc>

序列模式挖掘算法

- ❑ 算法要求：高效、可扩展、找出完全集、融入各类用户指定约束
- ❑ Apriori 性质仍然成立：若子序列 s_1 不频繁，则 s_1 的任何超序列都不可能频繁
- ❑ 代表性算法
 - ❑ GSP (Generalized Sequential Patterns): Srikant & Agrawal @ EDBT'96)

Sequential Pattern Mining Algorithms

- ❑ Algorithm requirement: Efficient, scalable, finding complete set, incorporating various kinds of user-specific constraints
- ❑ The Apriori property still holds: If a subsequence s_1 is infrequent, none of s_1 's super-sequences can be frequent
- ❑ Representative algorithms
 - ❑ GSP (Generalized Sequential Patterns): Srikant & Agrawal @ EDBT'96)

GSP: 基于 Apriori 的序列模式挖掘

- 初始候选: 全部 8 个单例序列
 - <a>, , <c>, <d>, <e>, <f>, <g>, <h>
- 扫描数据库一次, 为每个候选计数支持度

SID	序列
10	<(bd)cb(ac)>
20	<(bf)(ce)b(fg)>
30	<(ah)(bf)abf>
40	<(be)(ce)d>
50	<a(bd)bcb(ade)>

min_sup = 2

候选	sup
<a>	3
	5
<c>	4
<d>	3
<e>	3
<f>	2
<g>	1
<h>	1



GSP (Generalized Sequential Patterns): Srikant & Agrawal @ EDBT'96)

GSP: Apriori-Based Sequential Pattern Mining

- Initial candidates: All 8-singleton sequences
 - <a>, , <c>, <d>, <e>, <f>, <g>, <h>
- Scan DB once, count support for each candidate

SID	Sequence
10	<(bd)cb(ac)>
20	<(bf)(ce)b(fg)>
30	<(ah)(bf)abf>
40	<(be)(ce)d>
50	<a(bd)bcb(ade)>

min_sup = 2

Cand.	sup
<a>	3
	5
<c>	4
<d>	3
<e>	3
<f>	2
<g>	1
<h>	1



GSP (Generalized Sequential Patterns): Srikant & Agrawal @ EDBT'96)

GSP: 基于 Apriori 的序列模式挖掘

- 示例：生成长度为 2 的候选序列

单例 * 单例 – 合计: (6 * 6)

$min_sup = 2$

候选	sup
<a>	3
	5
<c>	4
<d>	3
<e>	3
<f>	2
<g>	1
<h>	1



	<a>		<c>	<d>	<e>	<f>
<a>	<aa>	<ab>	<ac>	<ad>	<ae>	<af>
	<ba>	<bb>	<bc>	<bd>	<be>	<bf>
<c>	<ca>	<cb>	<cc>	<cd>	<ce>	<cf>
<d>	<da>	<db>	<dc>	<dd>	<de>	<df>
<e>	<ea>	<eb>	<ec>	<ed>	<ee>	<ef>
<f>	<fa>	<fb>	<fc>	<fd>	<fe>	<ff>

集合 (无序) – 合计: (6*5) / 2

	<a>		<c>	<d>	<e>	<f>
<a>		<(ab)>	<(ac)>	<(ad)>	<(ae)>	<(af)>
			<(bc)>	<(bd)>	<(be)>	<(bf)>
<c>				<(cd)>	<(ce)>	<(cf)>
<d>					<(de)>	<(df)>
<e>						<(ef)>
<f>						

Apriori 剪枝

- 不剪枝 (包含 g 和 h):

$$8*8 + 8*7/2 = 92$$

长度为 2 的候选

- 剪枝后:

$$6*6 + 6*5/2 = 51$$

长度为 2 的候选

GSP: Apriori-Based Sequential Pattern Mining

- Example: Generate length-2 candidate sequences

singleton * singleton – Total: (6 * 6)

$min_sup = 2$

Cand.	sup
<a>	3
	5
<c>	4
<d>	3
<e>	3
<f>	2
<g>	1
<h>	1



	<a>		<c>	<d>	<e>	<f>
<a>	<aa>	<ab>	<ac>	<ad>	<ae>	<af>
	<ba>	<bb>	<bc>	<bd>	<be>	<bf>
<c>	<ca>	<cb>	<cc>	<cd>	<ce>	<cf>
<d>	<da>	<db>	<dc>	<dd>	<de>	<df>
<e>	<ea>	<eb>	<ec>	<ed>	<ee>	<ef>
<f>	<fa>	<fb>	<fc>	<fd>	<fe>	<ff>

Sets (unordered) – Total: (6*5) / 2

	<a>		<c>	<d>	<e>	<f>
<a>		<(ab)>	<(ac)>	<(ad)>	<(ae)>	<(af)>
			<(bc)>	<(bd)>	<(be)>	<(bf)>
<c>				<(cd)>	<(ce)>	<(cf)>
<d>					<(de)>	<(df)>
<e>						<(ef)>
<f>						

Apriori Pruning

- w/o pruning
(includes g and h):

$$8*8 + 8*7/2 = 92$$

length-2 candidates

- w/ pruning:

$$6*6 + 6*5/2 = 51$$

length-2 candidates

GSP 挖掘与剪枝

第5次扫描: 1 个候选, 1 个长度为 5 的序列模式
第4次扫描: 8 个候选, 7 个长度为 4 的序列模式
第3次扫描: 46 个候选, 20 个长度为 3 的序列模式, 20 个候选根本不在数据库中
第2次扫描: 51 个候选, 19 个长度为 2 的序列模式, 10 个候选根本不在数据库中
第1次扫描: 8 个候选, 6 个长度为 1 的序列模式

$6*6 + 6*5/2 = 51$

GSP 算法

- 重复, 从 k = 1 直到 k <= length
 - 扫描数据库以找出“长度为 k”的频繁序列
 - 由“长度为 k”的频繁序列用 Apriori 生成“长度为 (k+1)”的候选序列
 - 令 k = k+1
- 直到没有频繁序列或无法再生成候选

<(bd)cba>	长度
<abba> <(bd)bc> ...	4
<abb> <aab> <aba> <baa> <bab> ...	3
<aa> <ab> ... <af> <ba> <bb> ... <ff> <(ab)> ... <(ef)>	2
<a> <c> <d> <e> <f> <g> <h>	1

删除不在数据库中的候选或 < min_sup 的候选

SID	序列
10	<(bd)cb(ac)>
20	<(bf)(ce)b(fg)>
30	<(ah)(bf)abf>
40	<(be)(ce)d>
50	<a(bd)bcb(ade)>

min_sup = 2

GSP Mining and Pruning

5th scan: 1 cand. 1 length-5 seq. pat.
4th scan: 8 cand. 7 length-4 seq. pat.
3rd scan: 46 cand. 20 length-3 seq. pat. 20 cand. not in DB at all
2nd scan: 51 cand. 19 length-2 seq. pat. 10 cand. not in DB at all
1st scan: 8 cand. 6 length-1 seq. pat.

$6*6 + 6*5/2 = 51$

The GPS algorithm

- Repeat, starting at k = 1 until k <= length
 - Scan DB to find “length-k” frequent sequences
 - Generate “length-(k+1)” candidate sequences from “length-k” frequent sequences using **Apriori**
 - set k = k+1
- Until no frequent sequence or no candidate can be found

<(bd)cba>	length
<abba> <(bd)bc> ...	4
<abb> <aab> <aba> <baa> <bab> ...	3
<aa> <ab> ... <af> <ba> <bb> ... <ff> <(ab)> ... <(ef)>	2
<a> <c> <d> <e> <f> <g> <h>	1

Remove Candidates not in DB or Candidates < min_sup

SID	Sequence
10	<(bd)cb(ac)>
20	<(bf)(ce)b(fg)>
30	<(ah)(bf)abf>
40	<(be)(ce)d>
50	<a(bd)bcb(ade)>

min_sup = 2

模式挖掘：高级方法

- ❑ 挖掘多层与多维模式
- ❑ 基于约束的模式挖掘
- ❑ 挖掘序列模式
- ❑ 挖掘子图模式
- ❑ 小结



Pattern Mining: Advanced Methods

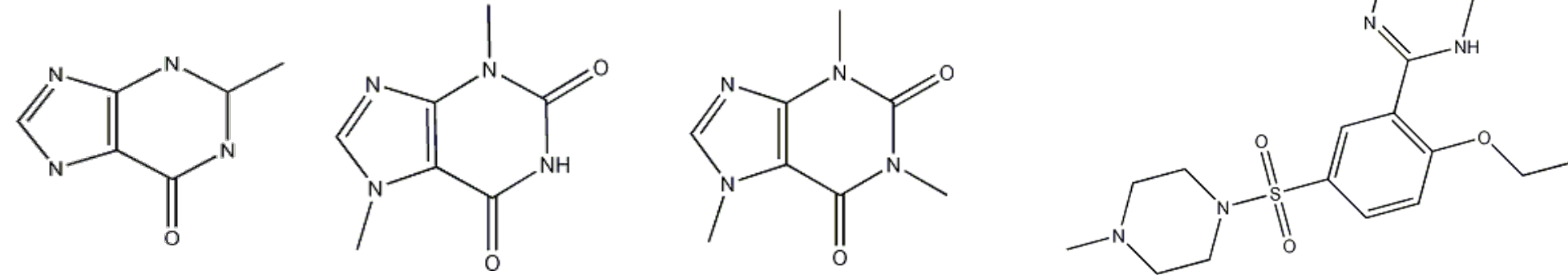
- ❑ Mining Multi-level and multi-dimension Patterns
- ❑ Constraint-based Pattern Mining
- ❑ Mining Sequential Patterns
- ❑ Mining Subgraph Patterns
- ❑ Summary



什么是图模式挖掘?

化学信息学:

挖掘频繁化合物结构



社交网络、网络社区、推文、...

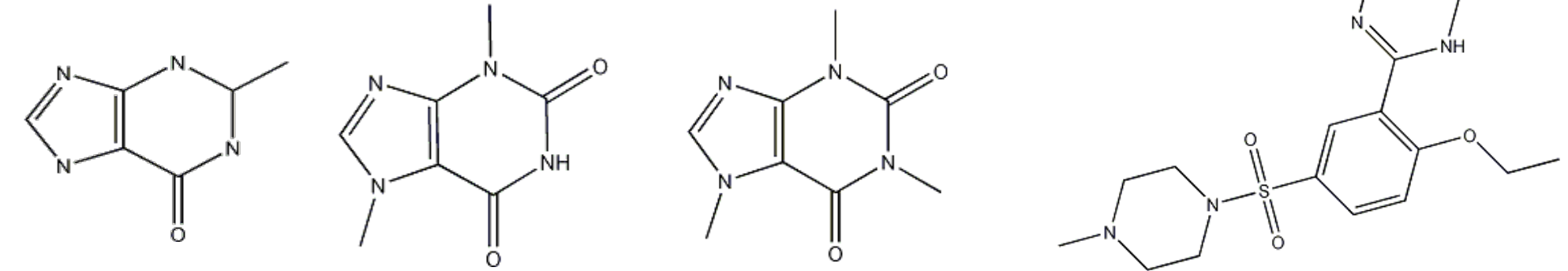
发现频繁的合作子图



What Is Graph Pattern Mining?

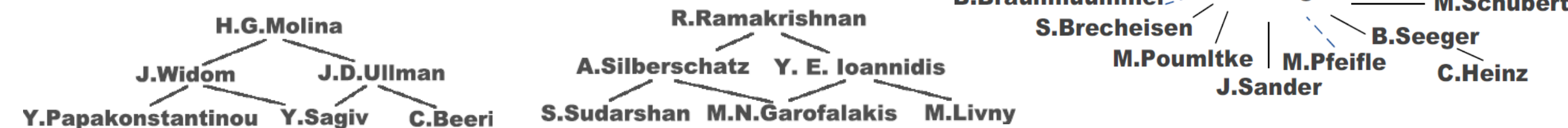
Chem-informatics:

Mining frequent chemical compound structures



Social networks, web communities, tweets, ...

Finding frequent research collaboration subgraphs



频繁 (子) 图模式

- 给定带标号的图数据集 $D = \{G_1, G_2, \dots, G_n\}$, 子图 g 的支持图集为 $D_g = \{G_i \mid g \subseteq G_i, G_i \in D\}$

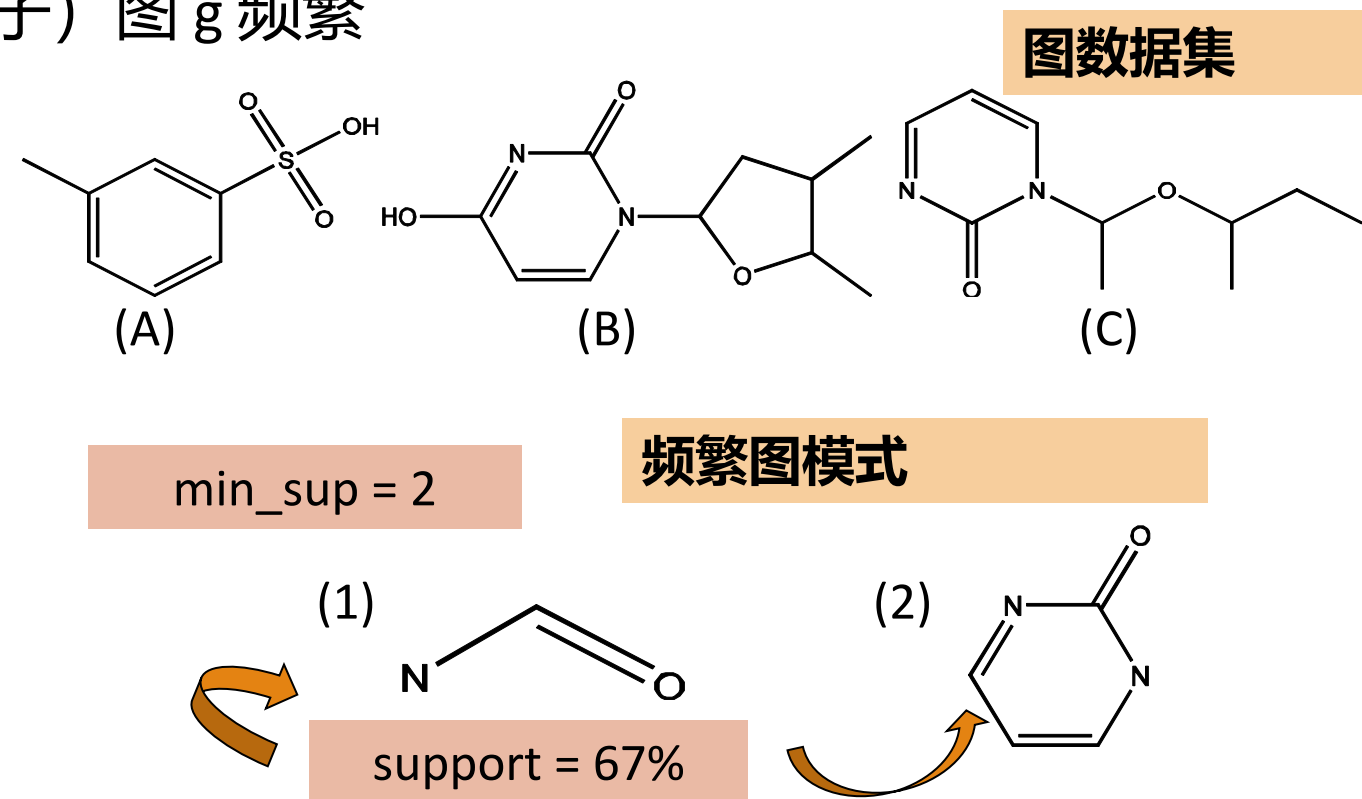
$\text{support}(g) = |D_g| / |D|$

- 若 $\text{support}(g) \geq \text{min_sup}$, 则 (子) 图 g 频繁

- 例: 化学结构

- 另一种情形:

- 从单个大图或网络中挖掘频繁子图模式



Frequent (Sub)Graph Patterns

- Given a labeled graph dataset $D = \{G_1, G_2, \dots, G_n\}$, the supporting graph set of a subgraph g is $D_g = \{G_i \mid g \subseteq G_i, G_i \in D\}$

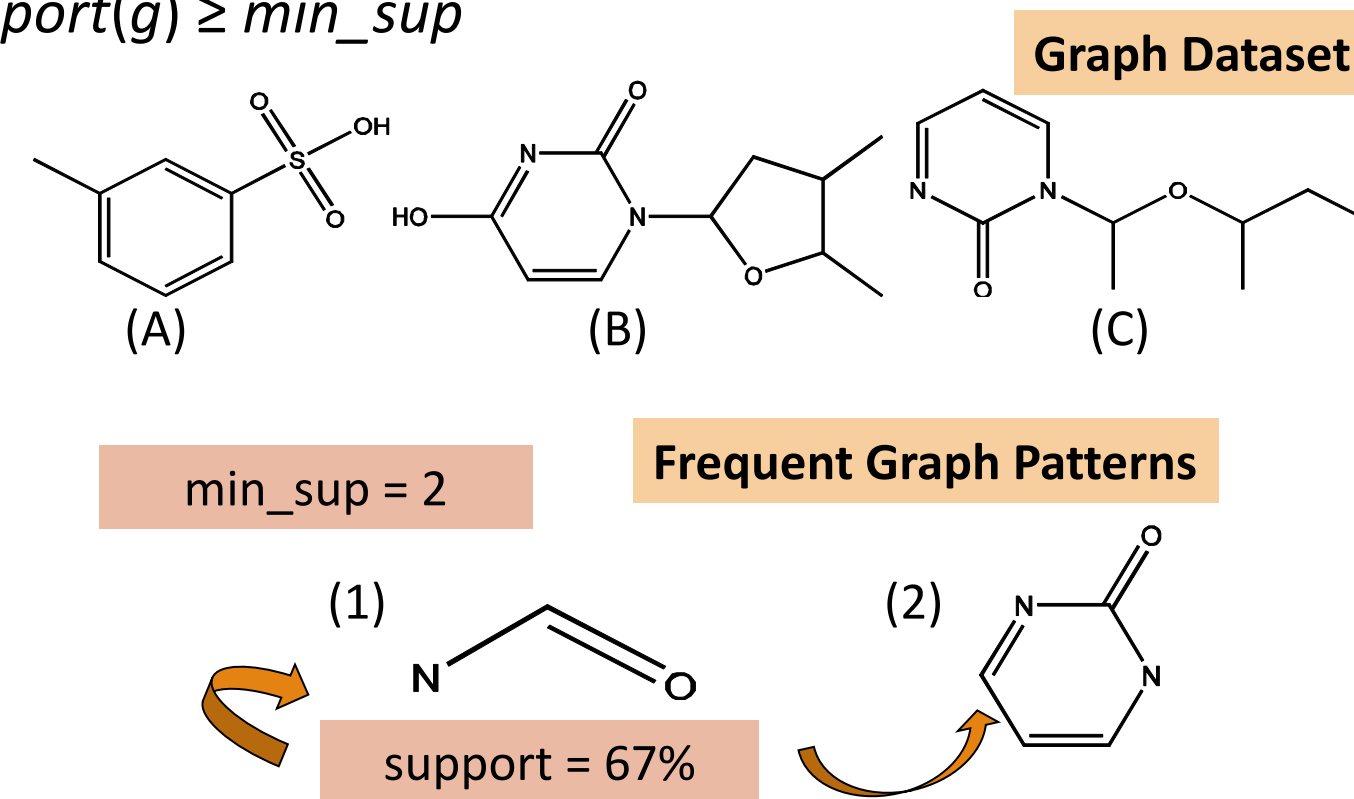
$\text{support}(g) = |D_g| / |D|$

- A (sub)graph g is **frequent** if $\text{support}(g) \geq \text{min_sup}$

- Ex.: Chemical structures

- Alternative:

- Mining frequent subgraph patterns from a single large graph or network



图模式挖掘的应用

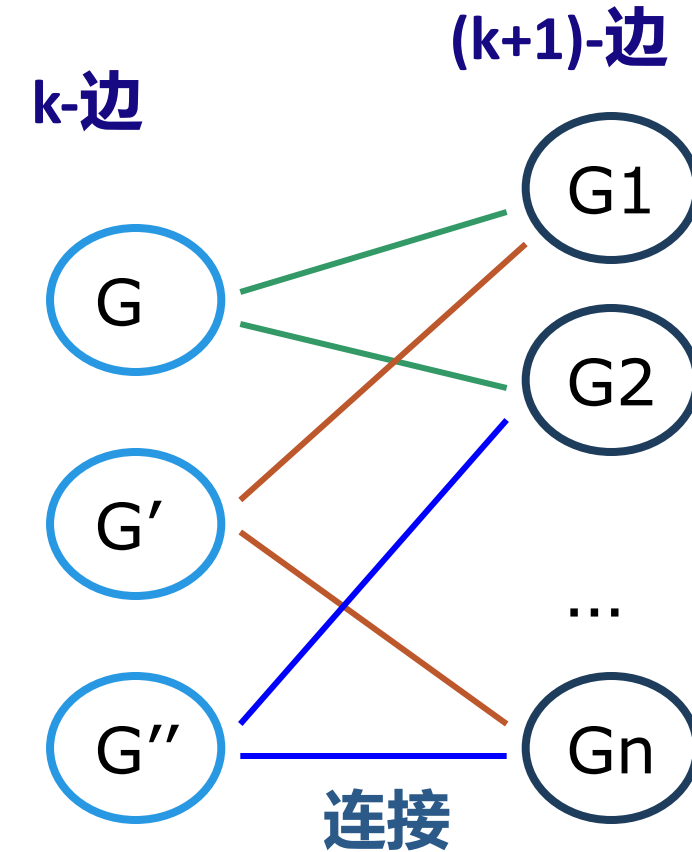
- ❑ 生物信息学
 - ❑ 基因网络、蛋白质相互作用、代谢通路
- ❑ 化学信息学：挖掘化合物结构
- ❑ 社交网络、网络社区、推文、...
- ❑ 手机网络、计算机网络、...
- ❑ Web 图、XML 结构、语义网、信息网络
- ❑ 软件工程：程序执行流分析
- ❑ 图分类、聚类、压缩、比较与相关分析的构建模块
- ❑ 图索引与图相似性搜索

Applications of Graph Pattern Mining

- ❑ Bioinformatics
 - ❑ Gene networks, protein interactions, metabolic pathways
- ❑ Chem-informatics: Mining chemical compound structures
- ❑ Social networks, web communities, tweets, ...
- ❑ Cell phone networks, computer networks, ...
- ❑ Web graphs, XML structures, Semantic Web, information networks
- ❑ Software engineering: Program execution flow analysis
- ❑ Building blocks for graph classification, clustering, compression, comparison, and correlation analysis
- ❑ Graph indexing and graph similarity search

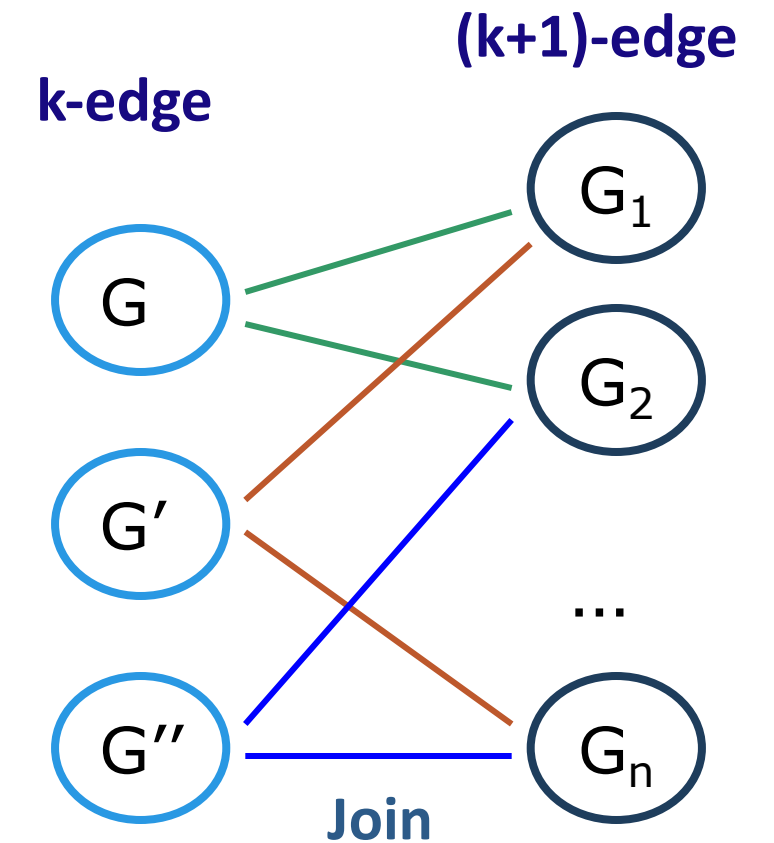
基于 Apriori 的方法

- Apriori 性质（反单调性）：大小为 k 的子图频繁，当且仅当其所有子图都频繁
- 若对应的两个 k -边/顶点子图都频繁，则生成大小为 $(k+1)$ 的边/顶点候选子图
- 迭代挖掘过程：
 - 候选生成 □ 候选剪枝 □ 支持度计数 □ 候选消除



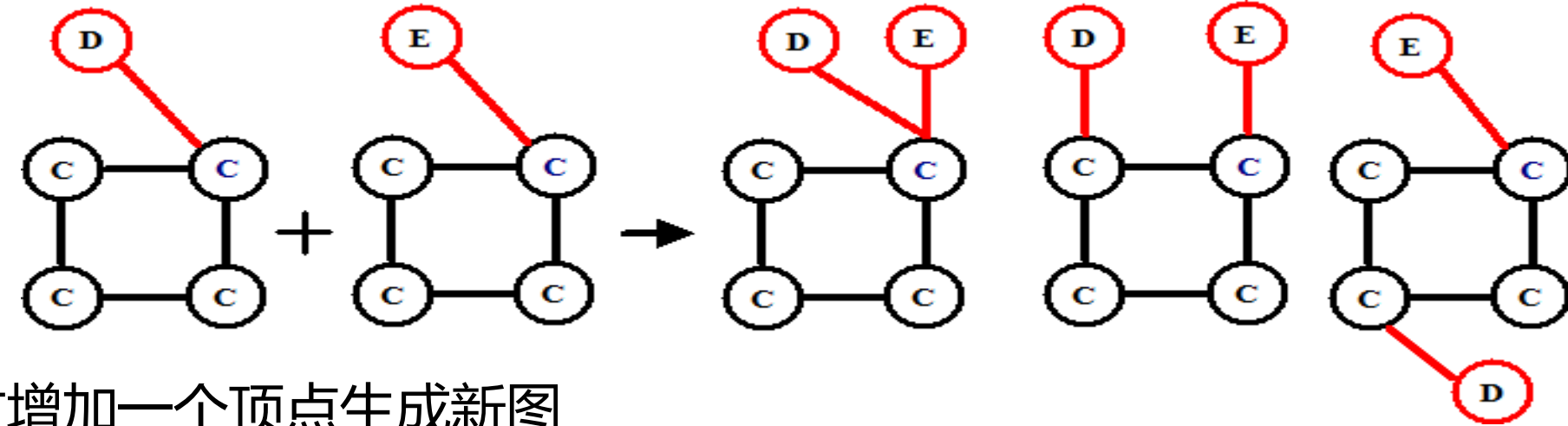
Apriori-Based Approach

- The Apriori property (anti-monotonicity): A size- k subgraph is **frequent** if and only if all of its **subgraphs are frequent**
- A candidate size- $(k+1)$ edge/vertex subgraph is generated if its corresponding two k -edge/vertex subgraphs are frequent
- Iterative mining process:
 - Candidate-generation → candidate pruning → support counting → candidate elimination



候选生成：顶点增长 vs. 边增长

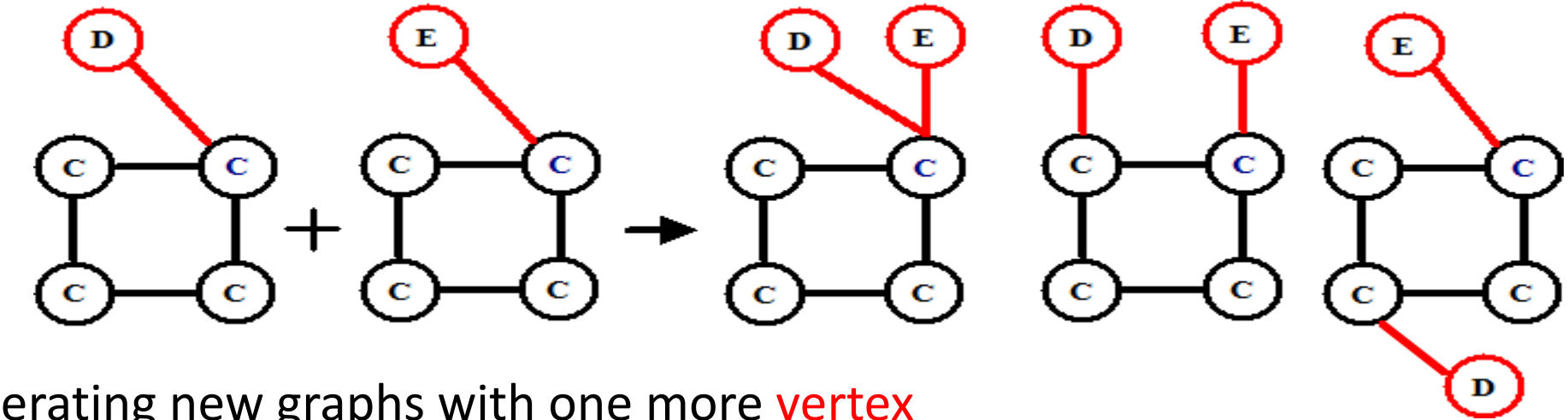
- 方法：广度搜索，用 Apriori 连接两个大小为 k 的图
- 生成大小为 $(k+1)$ 的候选图时可能性很多



- 通过增加一个顶点生成新图
 - AGM (Inokuchi, Washio, & Motoda, PKDD'00)
- 通过增加一条边生成新图
 - FSG (Kuramochi & Karypis, ICDM'01)
- 性能表明边增长更高效

Candidate Generation: Vertex Growing vs. Edge Growing

- Methodology: Breadth-search, Apriori joining two size- k graphs
- Many possibilities at generating size- $(k+1)$ candidate graphs



- Generating new graphs with one more **vertex**
 - AGM (Inokuchi, Washio, & Motoda, PKDD'00)
- Generating new graphs with one more **edge**
 - FSG (Kuramochi & Karypis, ICDM'01)
- Performance shows *via edge growing* is more efficient

模式增长方法

- 深度优先：由 k -边子图增长到 $(k+1)$ -边，再到 $(k+2)$ -边子图

- 主要挑战

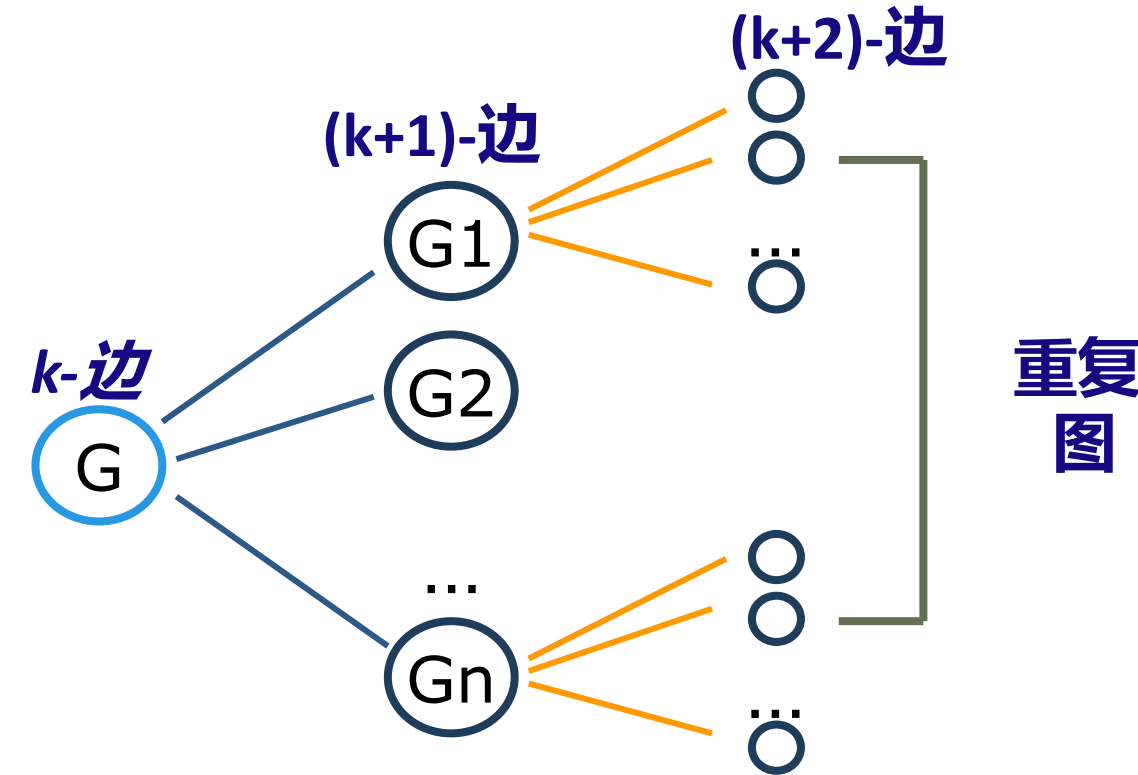
- 会生成许多重复子图

- 解决问题的主要思路

- 定义生成子图的次序

- DFS 生成树：用深度优先搜索将图展平为序列

- gSpan (Yan & Han, ICDM'02)



Pattern-Growth Approach

- Depth-first growth of subgraphs from k -edge to $(k+1)$ -edge, then $(k+2)$ -edge subgraphs

- Major challenge

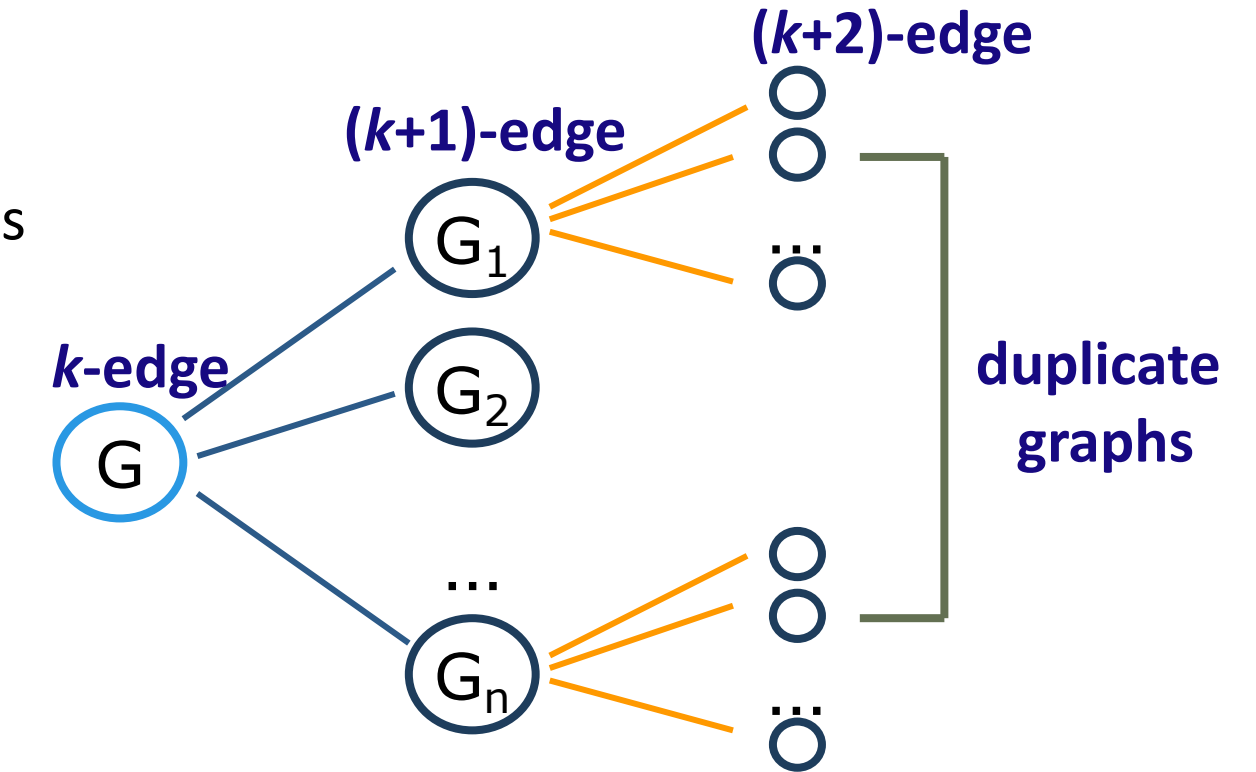
- Generating many duplicate subgraphs

- Major idea to solve the problem

- Define an order to generate subgraphs

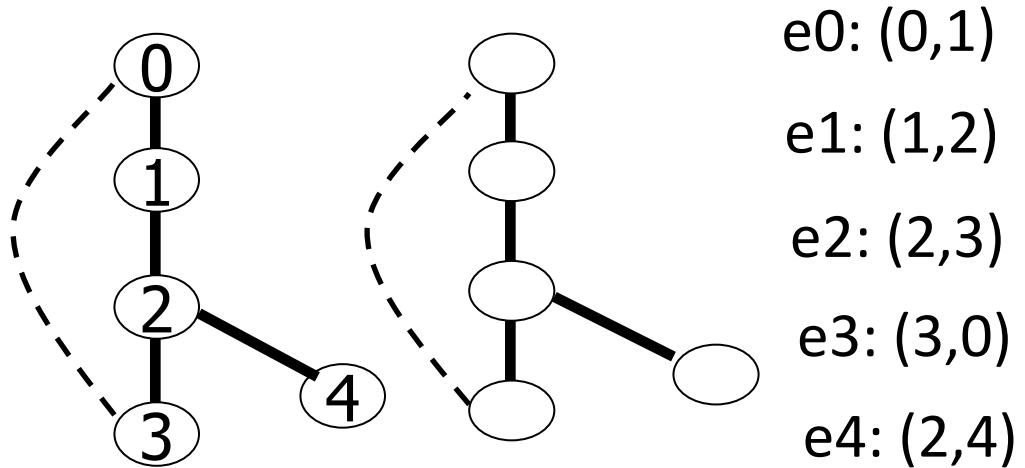
- DFS spanning tree: Flatten a graph into a sequence using depth-first search

- gSpan (Yan & Han, ICDM'02)



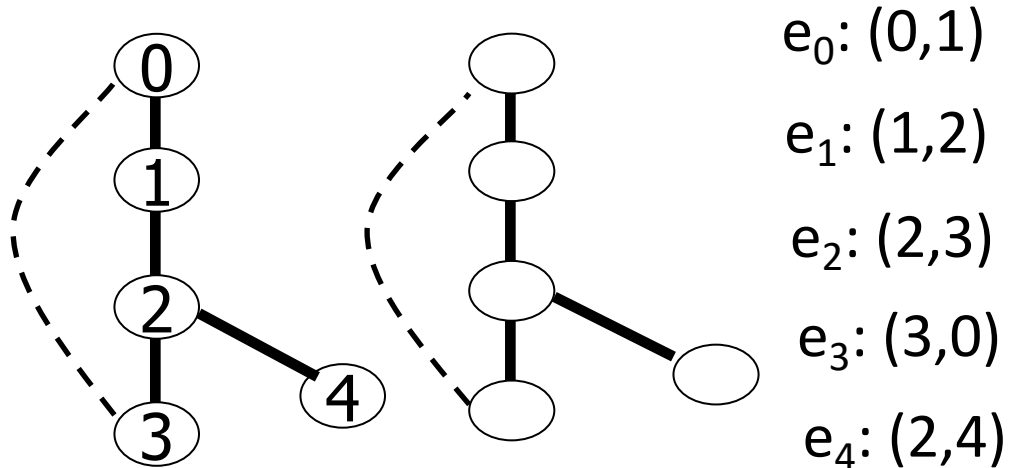
gSPAN: 按序进行图模式增长

- 子图模式增长中的最右路径扩展
 - 最右路径: 从根到最右叶子的路径 (每一步选择下标最小的顶点)
 - 减少重复子图的生成
- 完备性: 用最右路径扩展枚举图是完备的
- DFS 编码: 用深度优先搜索将图展平为序列



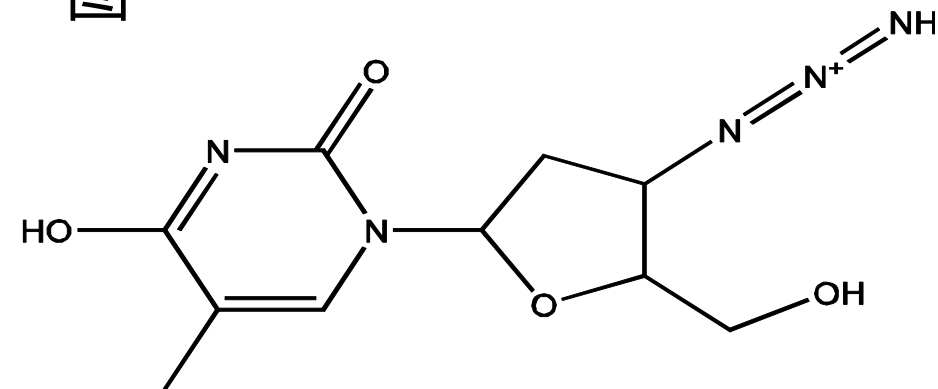
gSPAN: Graph Pattern Growth in Order

- Right-most path extension in subgraph pattern growth
 - Right-most path: The path from root to the right-most leaf (choose the vertex with the **smallest** index at each step)
 - Reduce generation of duplicate subgraphs
- Completeness: The enumeration of graphs using right-most path extension is complete
- DFS code: Flatten a graph into a sequence using depth-first search



挖掘闭图模式

- 2^n 个子图 -> 闭频繁子图
- 频繁图 G 是闭的，当且仅当不存在与 G 具有相同支持度的 G 的超图

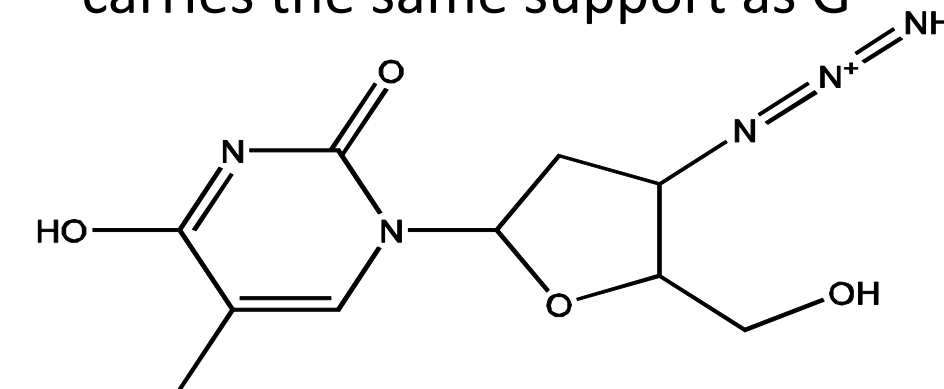


若该子图在图数据集中是闭的，则意味着其任何频繁超图都不具有相同支持度

- 无损压缩：不含非闭图，但仍保证挖掘结果完备
- 算法 CloseGraph：直接挖掘闭图模式

Mining Closed Graph Patterns

- 2^n subgraphs -> *closed frequent subgraphs*
- A frequent graph G is *closed* if there exists no supergraph of G that carries the same support as G

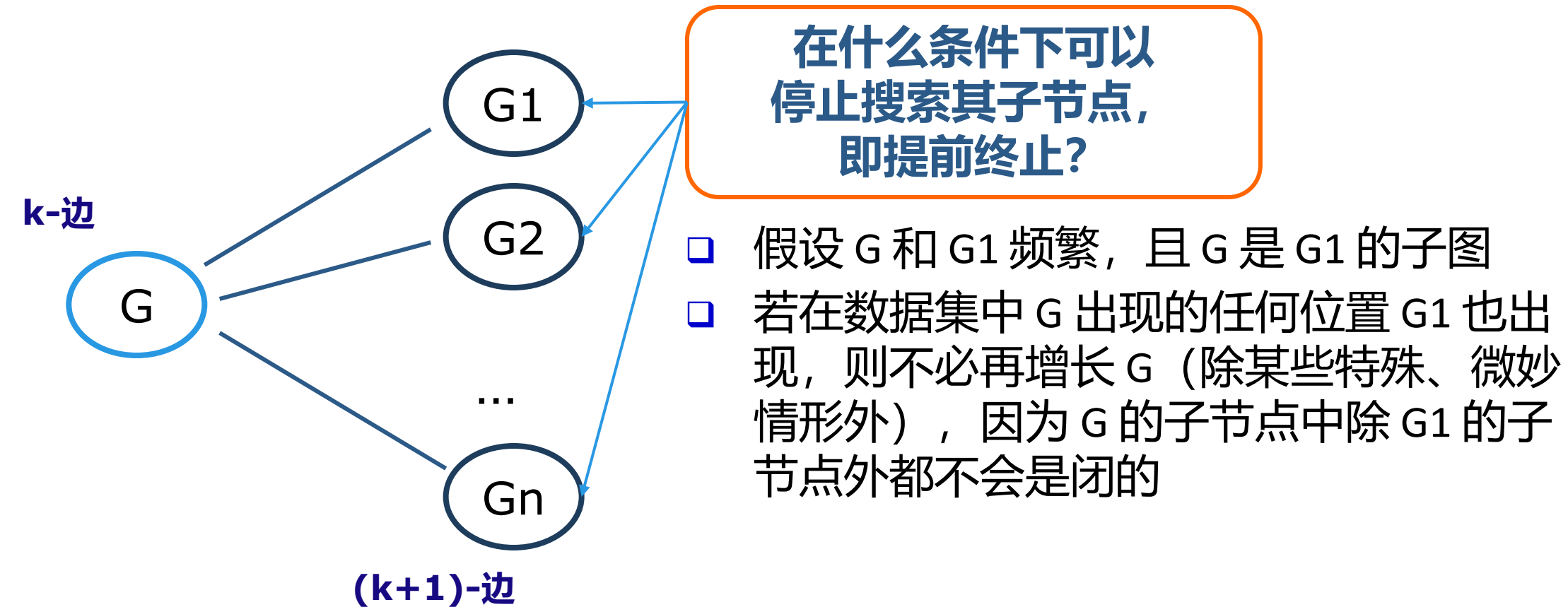


If this subgraph is *closed* in the graph dataset, it implies that none of its frequent super-graphs carries the same support

- *Lossless compression*: Does not contain non-closed graphs, but still ensures that the mining result is complete
- Algorithm CloseGraph: Mines closed graph patterns directly

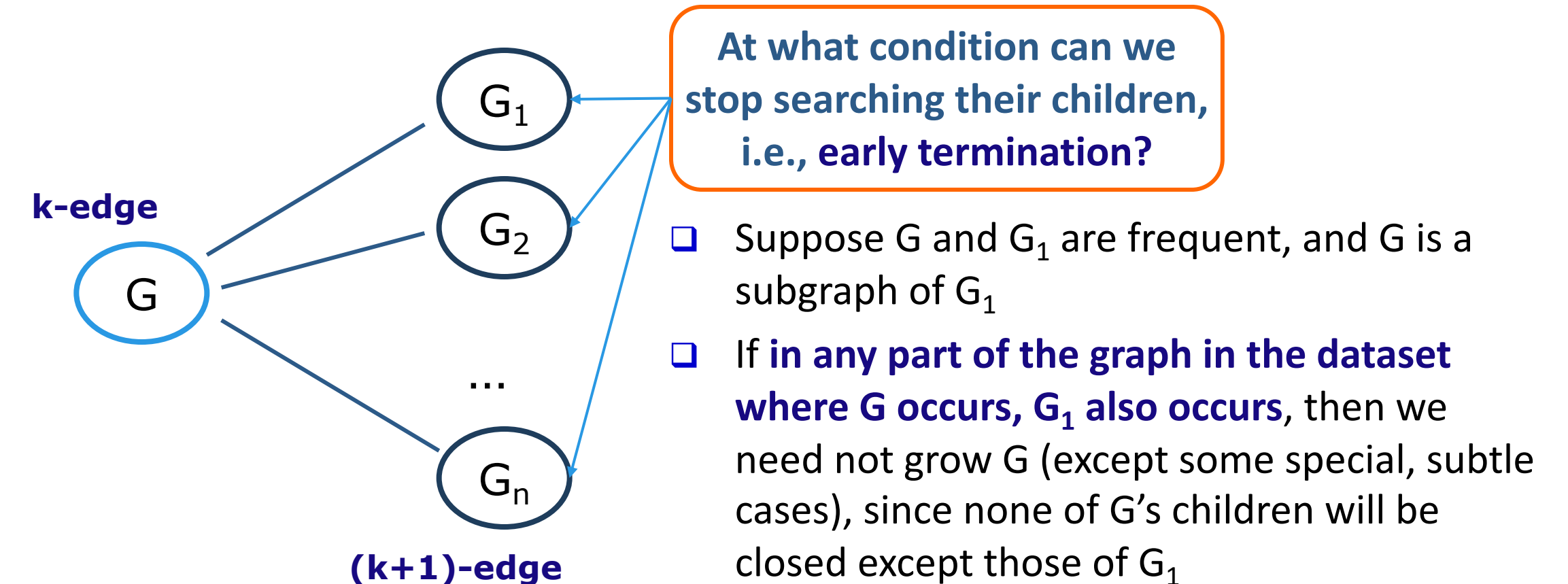
CloseGraph: 直接挖掘闭图模式

- CloseGraph: 通过扩展 gSpan 挖掘闭图模式 (Yan & Han, KDD'03)



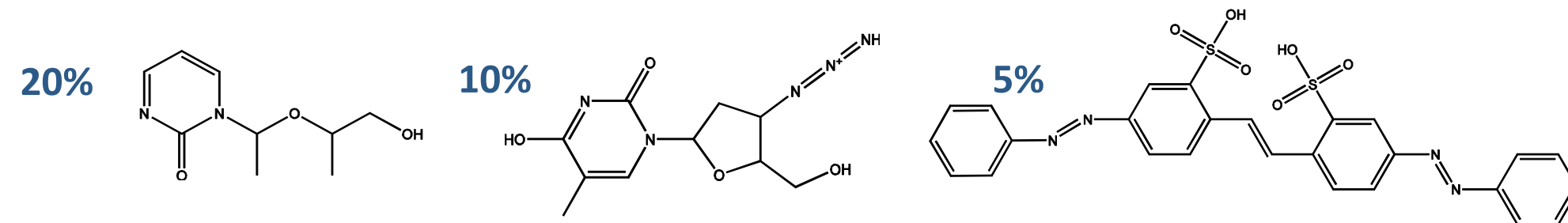
CloseGraph: Directly Mining Closed Graph Patterns

- CloseGraph: Mining closed graph patterns by extending gSpan (Yan & Han, KDD'03)

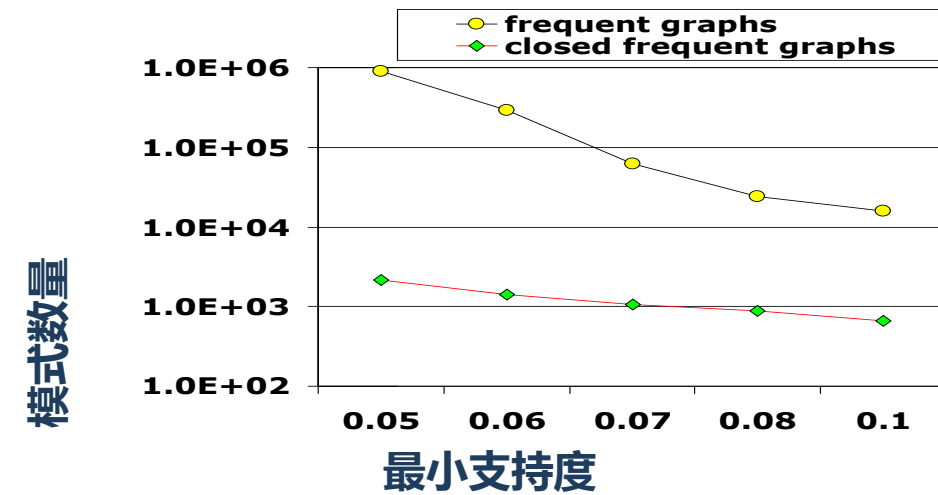


实验与性能比较

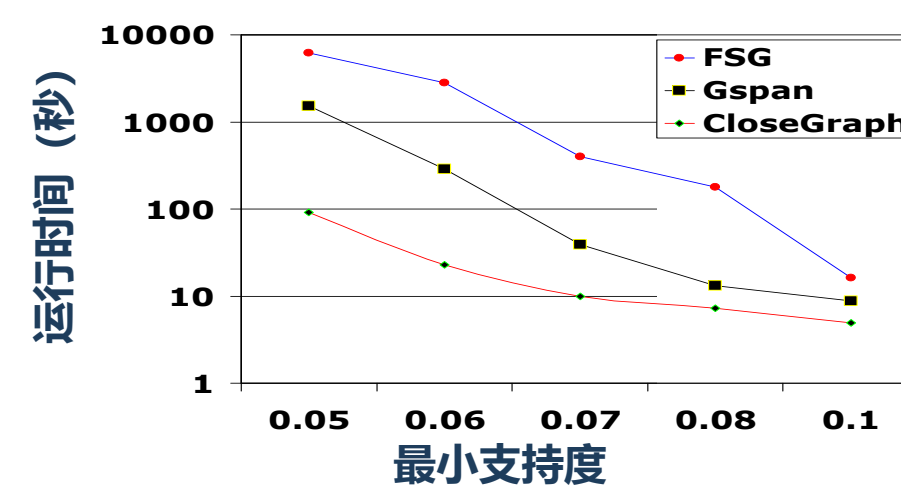
- 来自 NCI/NIH 的 AIDS 抗病毒筛选化合物数据集
- 该数据集包含 43,905 种化合物
- 发现的模式：最小支持度越小，发现的子图模式越大、越有趣



模式数量：频繁 vs. 闭

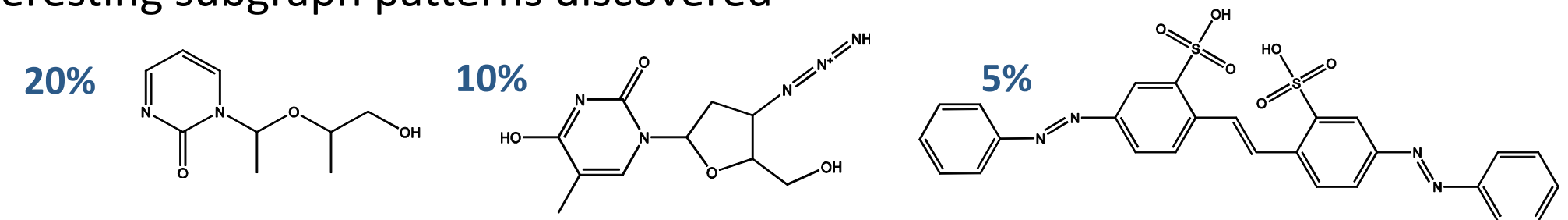


运行时间：频繁 vs. 闭

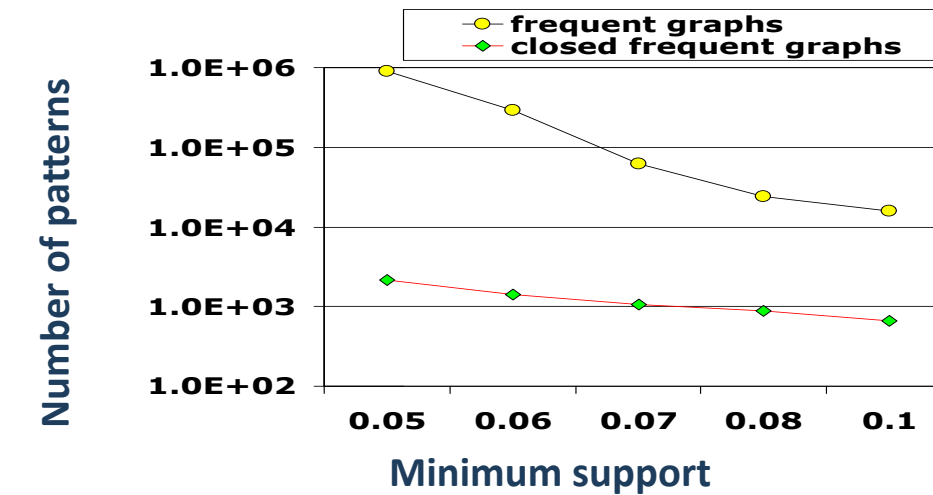


Experiment and Performance Comparison

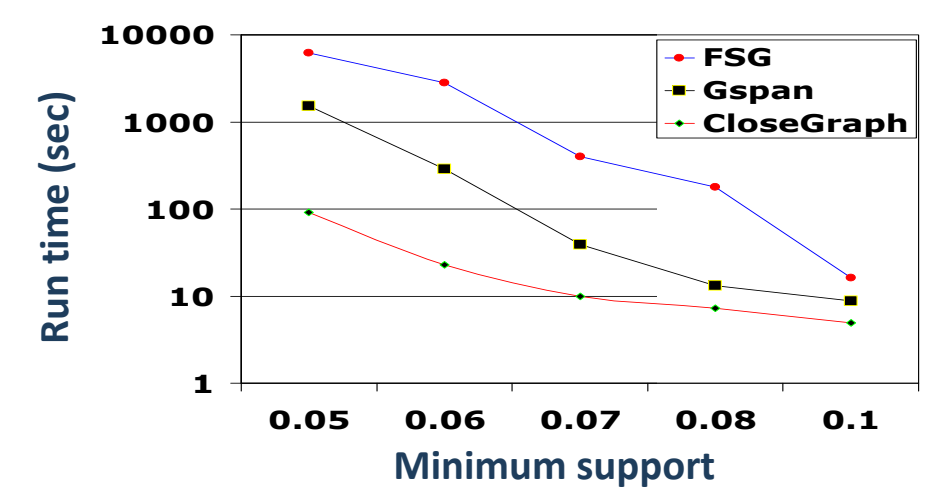
- The AIDS antiviral screen compound dataset from NCI/NIH
- The dataset contains 43,905 chemical compounds
- Discovered patterns: The smaller minimum support, the bigger and more interesting subgraph patterns discovered



of Patterns: Frequent vs. Closed



Runtime: Frequent vs. Closed

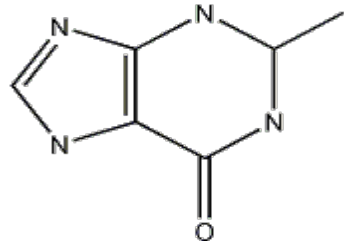


应用：支持子结构相似性搜索

- 在图数据库中查找含有与给定查询图相似的子结构的图

- 例：数据：化合物数据库

- 查询图 q:



- 如何高效进行相似性搜索？

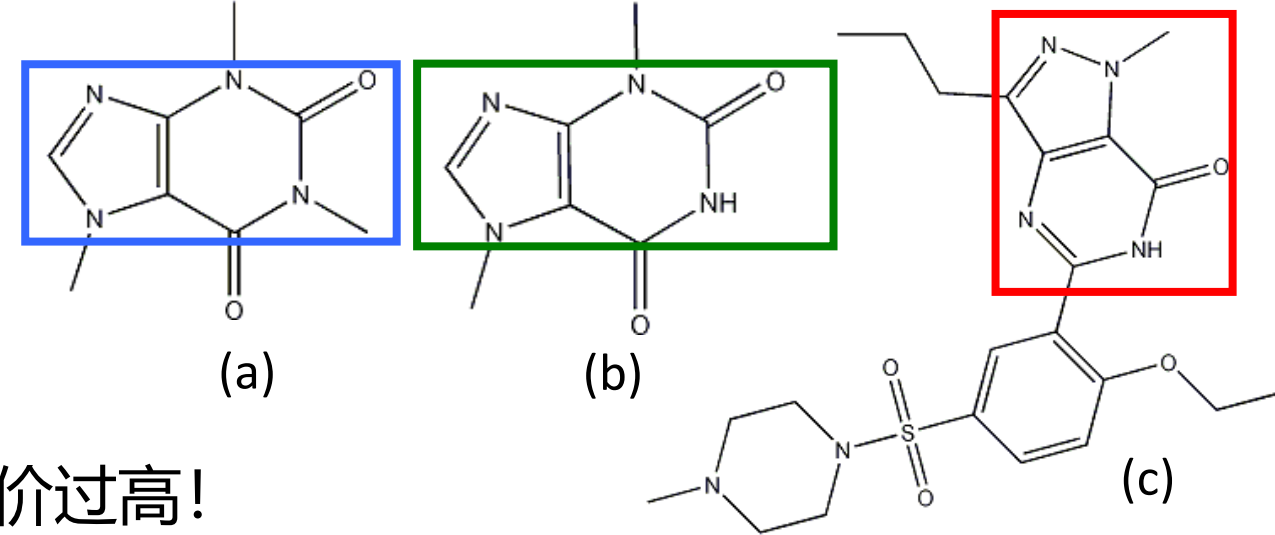
- 顺序扫描 + 计算子图相似度 – 代价过高！

- 构建图索引以支持近似搜索？

- 需要海量子图才能覆盖所有相似子图！

- 一个巧妙的方案 (Yan, Yu, & Han, SIGMOD'05):

- 保留图索引结构，但在查询空间中选择特征

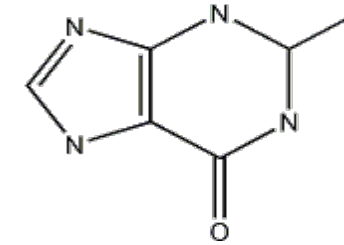


Application: Support Substructure Similarity Search

- Find graphs in a graph DB containing substructures similar to a given query graph

- Ex. Data: A chemical compound DB

- A query graph q:



- How to do similarity search efficiently?

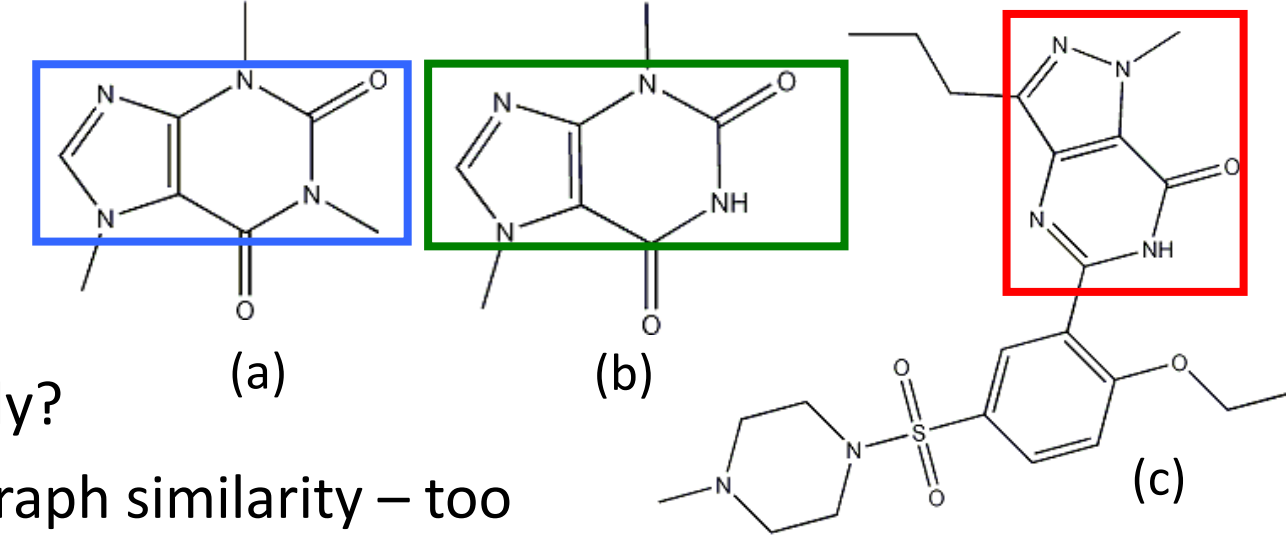
- Sequential scan + computing subgraph similarity – too costly!

- Build graph indices to support approximate search?

- Need an explosive number of subgraphs to cover all the *similar* subgraphs!

- An elegant solution (Yan, Yu, & Han, SIGMOD'05):

- Keep the graph index structure, but **select features** in the **query space**



小结

- ❑ 挖掘多样化模式
- ❑ 基于约束的频繁模式挖掘
- ❑ 序列模式挖掘
- ❑ 图模式挖掘

Summary

- ❑ Mining Diverse Patterns
- ❑ Constraint-Based Frequent Pattern Mining
- ❑ Sequential Pattern Mining
- ❑ Graph Pattern Mining

问答



香港科技大学(广州)
THE HONG KONG
UNIVERSITY OF SCIENCE AND
TECHNOLOGY (GUANGZHOU)

Q & A



香港科技大学(广州)
THE HONG KONG
UNIVERSITY OF SCIENCE AND
TECHNOLOGY (GUANGZHOU)